

GarageST Developer Handbook

Engineering Architecture & Implementation

Flutter + Spring Boot + PostgreSQL + Cloudflare R2 + Firebase Cloud Messaging

Living engineering reference · 2026-10-02 · TVASTRI

Source of truth: GARAGEST_DEVELOPER_HANDBOOK.md · Contains variable names only, no secrets

Living engineering reference for GarageST (Flutter app + Spring Boot backend). It describes the system **as implemented** after the 2026-10-02 push-notification implementation (on top of the 2026-09-30 workflow + trip/navigation change). Revision 2026-10-02. It never contains secrets — configuration is referred to by variable **name** only.

Legend — *Implemented* · *Legacy* (kept for compatibility, do not extend) · *Declared-only* (enum/column exists, nothing writes it) · *Not implemented* · *Config-dependent* · *Manual-only* (needs a device). Status terms used for newer sections: **IMPLEMENTED · PHYSICALLY VALIDATED · CONFIGURATION-DEPENDENT · PARTIALLY IMPLEMENTED · NOT IMPLEMENTED · FUTURE / ROADMAP · LEGACY** (*Implemented* = IMPLEMENTED, *Manual-only* = awaiting physical validation, *Declared-only* = NOT IMPLEMENTED).

Repos: Flutter `D:\garagest_flutter` · Backend `D:\garageos-backend` · Production API `https://garagest-backend.onrender.com`.

Related documents: `GARAGEST_MEDIA_ARCHITECTURE.md` (media history/phases), `GARAGEST_NOTIFICATIONS.md` (notification quick reference; this handbook §28A is authoritative), Flutter `NOTIFICATIONS.md`, `CLAUDE.md` (per-repo agent rules). The previous revision is preserved in `docs/history/`.

Table of contents

1. Project Overview 4
2. Repository Structure 4
3. Technology Stack 4
4. System Architecture 4
5. Flutter Architecture 5
6. Backend Architecture 5
7. Security 5
8. Who Owns the State 6
9. State Machines 7
10. JobCard vs RepairTask 9
11. Customer Approval vs Manager Confirmation 9
12. Guided Workflow Engine 10
13. Backend Service Ownership 10
14. Workflow Coupling Rules 12
15. Booking Architecture 13
16. Pickup Architecture (deep) 13
17. Delivery Architecture (deep) 14
18. Delivery Location 14
19. Navigation Architecture 15
20. OTP Architecture 15
21. Trip ↔ Booking ↔ JobCard Relationships 16
22. Customer Journey Logic 16
23. Media Architecture 16
24. Trip Media vs JobCard Media 17
25. Database Architecture 17
26. Flyway Migrations 19
27. API Reference (business-critical) 19
28. WebSocket Architecture 22
28A. Push Notification System 22
29. Flutter Screen / Route Map 30
30. Key Code Paths 33
31. Sequence Diagrams 34
32. Error Handling 35

33. Transaction Boundaries 36
34. Legacy Compatibility Matrix 37
35. Deployment Runbook 38
36. Render, Cloudflare R2 and Environment 38
37. Testing Strategy and Test Evidence 39
38. Manual Test Matrix 40
39. Debugging Runbook 42
40. Architectural Rules — DO NOT BREAK THESE 43
41. Architecture Decision Records 43
42. Change Log 45
43. How to Maintain This Handbook 45

1. Project Overview

GarageST is a smart garage/workshop management platform. It runs a vehicle-service garage end to end: a customer books a service (optionally with pickup), the vehicle is picked up where requested, then inspected, estimated, approved by the customer, repaired, quality-checked, invoiced, paid and delivered, and the job is closed. Drivers move the vehicle with live navigation, OTP handover and photo evidence; **proactive push notifications** tell the right person about the important steps.

- **Users (roles):** Owner, Manager, Service Advisor, Technician, Driver, Accountant, Cashier, Customer (`RoleCode` on the backend, `UserRole` in Flutter; Flutter intentionally omits `SUPER_ADMIN`/`USER`).
- **Capabilities (IMPLEMENTED):** booking, job-card workflow (inspection → estimate → approval → repair → QC → invoice → payment → delivery → close), pickup and delivery trips with live map and routing, OTP-verified handover, evidence photos/videos, customer portal with live vehicle journey, **role-aware push notifications (32 events), in-app notification history with unread badge and deep links** (§28A), garage-scoped users with multi-garage owner views.
- **Not implemented:** online payment gateway (payment is recorded, not collected online), SMS/WhatsApp/e-mail messaging (the password-reset sender is a logging stub), iOS push, analytics/AI features.
- **Runtime:** Flutter (Android is the tested target; iOS parked) → REST `/api/v1` + STOMP `/ws` (live trip only) → Spring Boot → PostgreSQL. Media bytes go straight to Cloudflare R2. Push: Spring Boot → Firebase Cloud Messaging → device. Hosting: Render (Docker).

2. Repository Structure

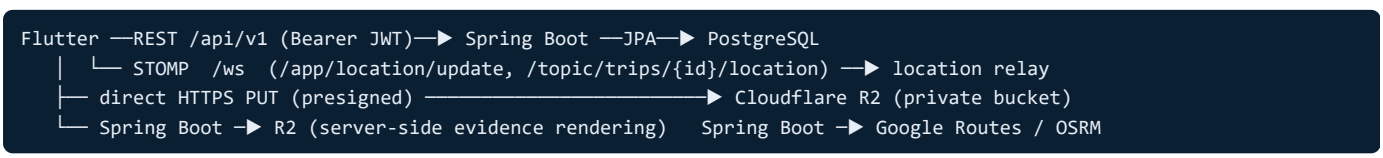
Repo	Layout
Flutter	`lib/app` (composition root `app.dart`, `routes/`, splash) · `lib/core` (ApiClient, EnvConfig, secure storage, location/geocoding, utils) · `lib/features/<feature>/{models,services,screens,state,widgets}` · `lib/shared` · `android/app/src/main/kotlin/com/garagest/app` (`MainActivity`, `EvidenceVideoProcessor`) · `test/` mirrors `lib/`
Backend	`src/main/java/com/garageos/core` (enums, exceptions, security, audit base) · `modules/<module>/{controller,service(+impl),repository,entity,dto,mapper,validator}` · `src/main/resources/db/migration` (Flyway) · `Dockerfile`

Flutter features that matter here: `workflow`, `jobcard`, `estimate`, `repair`, `booking`, `navigation`, `handover`, `media`, `customer_portal`, `auth`, `notification` (plus `lib/core/push/`). Backend modules: `serviceworkflow`, `jobcard`, `estimate`, `estimateitem`, `repairtask`, `jobassignment`, `qualitycheck`, `invoice`, `delivery`, `booking`, `navigation`, `handover`, `media`, `customer`, `audit`, `identity`, `notification`.

3. Technology Stack

Only technologies present in the code: **Java 17**, **Spring Boot 4.0.7** (web, security, data-jpa, websocket, validation), **PostgreSQL** + **Flyway** (`spring.jpa.hibernate.ddl-auto=validate`), springdoc-openapi, AWS SDK S3 client (Cloudflare R2), Google Drive client (legacy media), **Docker** (`eclipse-temurin:17-jdk`, builds with `./mvnw clean package -DskipTests`), **Render**. **Flutter** (Dart `>=3.3.0 <4.0.0`), `provider`, `flutter_map` + `latlong2`, `google_maps_flutter`, `geolocator`, `geocoding`, `stomp_dart_client`, `video_player`, `image_picker`, `cached_network_image`, `flutter_cache_manager`. Native Android video: **Media3 Transformer 1.4.1** (declared in `android/app/build.gradle.kts`). Routing: Google Routes (when configured) with OSRM fallback. Push notifications: **Firebase Cloud Messaging** via **Firebase Admin SDK 9.9.0** (backend) and `firebase_core`, `firebase_messaging`, `flutter_local_notifications` + the Google Services Gradle plugin (Flutter/Android). **No Kafka or other message broker** — a PostgreSQL transactional outbox is used (§28A).

4. System Architecture



Responsibility split: the backend owns business state, validation and authorization; Flutter renders that state and never invents it; R2 owns media bytes; PostgreSQL is the source of truth.

Notification path (added 2026-10-02, §28A):

```
Business service ──(same transaction)──▶ notification_outbox ──▶ NotificationOutboxWorker ──▶ recipients + notification + notification_delivery
                                                                                                  └─ FcmSender (outside any DB transaction) ─▶ Firebase Cloud Messaging ─▶ Android
Android: foreground ─▶ local notification │ background/terminated ─▶ system tray │ tap ─▶ role-aware deep link ─▶ GarageST screen
```

WebSocket/STOMP is **not** used for notifications; it remains responsible only for live-trip location. Notifications do not replace the live-trip architecture.

5. Flutter Architecture

- **Composition root:** `lib/app/app.dart` (`GarageSTApp`, one `MultiProvider`). Services take `ApiClient` in their constructor and are registered once; screens use `context.read/watch`. No screen constructs an `ApiClient`.
- **Routing:** `Routes.*` constants in `app/routes/app_routes_data.dart`; builders in `app/routes/app_router.dart`. Some screens are pushed directly (`TripEvidenceCaptureScreen`, `LocationPickerScreen`, `RegisterScreen`).
- **State:** `provider` only. `AuthState`, `WorkflowState` (deliberately a mutable bag — legacy JS port), `MediaUploadQueueService`, `LiveTripState`. Feature-local UI state stays in `StatefulWidget`s.
- **Personas:** splash redirect by role → `EmployeeHomeScreen` (Dashboard/Jobs/Customers/Vehicles tabs + "+ Workflow" FAB; technician/driver are role branches of the dashboard), `OwnerHome`, `CustomerHomeScreen`, `DriverHomeScreen`.
- **Session:** tokens in `SecureStorageService` only. Role gating in `Permissions` is presentation only (§7).
- **Notifications:** `PushNotificationService` (registered `lazy:false` in `app.dart`), `NotificationNavigationService` (owns the app-wide `navigatorKey` and route observer), `NotificationState` and `NotificationService`; `FirebaseBootstrap.initialize()` runs in `main()`; `AuthState.onBeforeLogout` is the single logout hook. Code under `lib/core/push/` and `lib/features/notification/` (§28A).

6. Backend Architecture

`Controller → Service (interface + Impl) → Repository → Entity`, DTOs + MapStruct mappers. Cross-module rules of thumb: a module writes its own aggregate's status; where several services write `JobCard.status` they all pass through `JobCardStatusValidator`. Orchestration lives in `ServiceWorkflowServiceImpl` (thin: it delegates to the owning services). Garage scoping is enforced in services, not only by URL. The notification module (`modules/notification`) follows the same layout; its worker runs on a dedicated executor, and business services reach it only through `NotificationEventPublisher` / `NotificationEvents` (field-injected, optional).

7. Security

Authentication: JWT bearer (access token ~1 h, refresh ~7 d; configured under `security.jwt.*`). `/api/v1/auth/**`, `/api/v1/health/**` and static frontend paths are public (`SecurityConfig`); everything else needs authentication. Flutter never auto-refreshes (known gap, CLAUDE.md §5).

Authorization layers (all server-side):

Layer	Where	Examples
Role	`@PreAuthorize` on controllers	`WORKFLOW/JOBCARD/DELIVERY/ASSIGNMENT_OPERATIONAL_ROLES` = MANAGER, SERVICE_ADVISOR, OWNER; `acceptInvoice` = CUSTOMER
Garage scoping	services	`JobCardServiceImpl.authorizeJobCardGarage`, `DeliveryServiceImpl.authorizeDeliveryAction`, `NavigationTripServiceImpl.requireOperationalStaffOfGarage`, `NavigationRequestServiceImpl`

Layer	Where	Examples
Manager gate	`JobCardServiceImpl.authorizeProceedToRepair` (MANAGER only); `EstimateServiceImpl.authorizeEmployeeEstimateApproval` (MANAGER only, employee approval path)	repair start
Technician scope	`RepairTaskServiceImpl.authorizeRepairTaskAction`	technician may act only on their own non-cancelled assignment in their garage
Driver ownership	`NavigationTripServiceImpl.requireCallerIsDriver` / `getDriverTrip`	the `?driverId=` parameter must equal the authenticated user; driver must be an active DRIVER of the garage (`requireDriverOfGarage`)
Customer ownership	`HandoverServiceImpl`, `NavigationTripAccessGuard.authorizeViewer`, `EstimateServiceImpl.getOwnedCustomerEstimate`, portal services	not-found instead of forbidden, so existence is not leaked
OTP	`HandoverServiceImpl`	see §20
Media	`R2MediaStorageProvider`	private bucket, short-lived presigned URLs, server-generated keys
Notifications	`NotificationController`, `DeviceTokenController`, `NotificationRecipientResolver`	identity from the JWT only; history/mark-read scoped to the caller; device tokens reassigned/deactivated only for the owner; staff recipients garage-scoped (§28A.6, §28A.14)

What Flutter UI gating does NOT protect: hiding a button (`Permissions.*`, `WorkflowStepDefinition.roles`, `allowedActions`) is cosmetic. Every rule that matters is re-checked by the backend; never rely on the UI to prevent an action. Several workflow endpoints have no `@PreAuthorize` and rely on service-level checks — do not assume a role check exists on a controller method without reading the service.

Secrets: environment variables only (§36). Known limitation: `application.properties` contains a default JWT secret value in source control, and `src/test/resources/application-test.properties` contains a local test DB password — move/rotate both (values are deliberately not reproduced here).

8. Who Owns the State

State	Owner (only writer of record)	Authorization of the transition
`JobCard.status`	`JobCardServiceImpl` for most; also `EstimateServiceImpl` (`WAITING_FOR_APPROVAL`, `REPAIR_PENDING`), `RepairTaskServiceImpl` / `JobAssignmentServiceImpl.completeJob` (`REPAIR_COMPLETED`), `ServiceWorkflowServiceImpl.performQualityCheck` (`QUALITY_CHECK`), `QualityCheckServiceImpl`, `InvoiceServiceImpl`, `DeliveryServiceImpl` (`DELIVERED`). All call `JobCardStatusValidator.validate`.	per operation (§13)
`Estimate.status`	`EstimateServiceImpl` (`DRAFT`, `APPROVED`, `REJECTED`), `EstimateItemServiceImpl` (`WAITING_FOR_APPROVAL` when totals are recalculated)	customer owner for approve/reject
`RepairTask.status`	`RepairTaskServiceImpl`, `JobAssignmentServiceImpl` (start/complete of the linked task)	assigned technician or operational staff
`JobAssignment.status`	`JobAssignmentServiceImpl` (+ `RepairTaskServiceImpl.syncLinkedJobAssignment`)	assignee / assignment roles
`Booking.status`	`BookingServiceImpl`; `JobCardServiceImpl.createJobCard` sets `COMPLETED`	customer (cancel), staff (confirm/reject)
`NavigationRequest.status`	`NavigationRequestServiceImpl` (`REQUESTED`), `NavigationTripServiceImpl` (`ASSIGNED`, `IN_PROGRESS`, `COMPLETED`)	garage staff (assign), driver (start/complete)
`NavigationTrip.status/currentLeg/arrivedAt`	`NavigationTripServiceImpl`	the driver (`requireCallerIsDriver`)
`VehicleHandover.status`	`HandoverServiceImpl`	customer generates, driver verifies

State	Owner (only writer of record)	Authorization of the transition
`Delivery.status`	`DeliveryServiceImpl` (only `DELIVERED` is written)	operational roles (manual) or the driver's verified trip
workflow `completedSteps` / `nextStep` / `progress`	`ServiceWorkflowServiceImpl.getWorkflowStatus` (pure projection of `JobCardStatus`)	read-only
Flutter `currentStep`	`WorkflowController` (derives from backend `completedSteps`)	none — presentation

9. State Machines

9.1 JobCard (`JobCardStatusValidator` is the single rule set)

Current	Allowed next	Trigger / API	Actor	Owner method	Validation	Auto?
`OPEN`	`INSPECTION_PENDING`, `CANCELLED`	`POST /workflow/{n}/inspection/start` (Flutter chains it after create)	staff	`JobCardServiceImpl.startInspection` (via `ServiceWorkflowServiceImpl.startInspection`)	transition	manual (client chains)
`INSPECTION_PENDING`	`INSPECTION_COMPLETED`, `CANCELLED`	`POST /workflow/{n}/inspection/complete` (one request per complaint)	staff	`JobCardServiceImpl.completeInspection`	inspections count = complaints	manual
`INSPECTION_COMPLETED`	`ESTIMATE_PENDING`, `CANCELLED`	`POST /workflow/{n}/estimate`	staff	`JobCardServiceImpl.prepareEstimate`	transition	manual
`ESTIMATE_PENDING`	`WAITING_FOR_APPROVAL`, `CANCELLED`	`PUT /estimates/{id}` ("finish")	staff	`EstimateServiceImpl.updateEstimate` (only when job is `ESTIMATE_PENDING`)	transition	manual
`WAITING_FOR_APPROVAL`	`REPAIR_PENDING`, `CANCELLED`	`PUT /estimates/{id}/approve`	**customer**	`EstimateServiceImpl.approveEstimateCanonical`	ownership; estimate not already approved	manual (customer)
`REPAIR_PENDING`	`REPAIR_IN_PROGRESS`, `CANCELLED`	`POST /workflow/{n}/repair/start` (= `/jobcards/{n}/repair/start`)	**manager**	`JobCardServiceImpl.startRepair`	MANAGER role, estimate `APPROVED`, transition; idempotent	manual (**manager gate**)
`REPAIR_IN_PROGRESS`	`REPAIR_COMPLETED`, `CANCELLED`	last RepairTask completes	technician	`RepairTaskServiceImpl.completeRepair` / `JobAssignmentServiceImpl.completeJob`	all tasks `COMPLETED`; job must be `REPAIR_IN_PROGRESS`	**automatic**
`REPAIR_COMPLETED`	`QUALITY_CHECK`, `REPAIR_PENDING`, `CANCELLED`	`POST /workflow/{n}/quality-check`	manager/staff	`ServiceWorkflowServiceImpl.performQualityCheck`	transition	manual
`QUALITY_CHECK`	`READY_FOR_INVOICE`, `REPAIR_PENDING` (QC fail / rework)	`POST /jobcards/{n}/quality-check/pass` / `/fail`	staff	`QualityCheckServiceImpl`	QC body	manual
`READY_FOR_INVOICE`	`INVOICE_GENERATED`, `CANCELLED`	`POST /workflow/{n}/invoice`	staff	`InvoiceServiceImpl.generateInvoice`	transition	manual

Current	Allowed next	Trigger / API	Actor	Owner method	Validation	Auto?
`INVOICE_GENERATED`	`READY_FOR_DELIVERY`, `CANCELLED`	`POST /workflow/{n}/payment`	staff/customer	`InvoiceServiceImpl.receivePayment`	invoice accepted; customer ownership	manual
`READY_FOR_DELIVERY`	`DELIVERED`, `CANCELLED`	**delivery trip completion** (primary) or `POST /deliveries` (garage collection only)	driver / staff	`DeliveryServiceImpl.completeDeliveryFromTrip` / `createDelivery`	invoice `GENERATED` + `PAID`; see §17	automatic on trip complete
`DELIVERED`	`CLOSED`	`PUT /jobcards/{n}/close` (`POST /workflow/{n}/close`)	operational roles	`JobCardServiceImpl.closeJobCard`	transition	manual
`CLOSED`, `CANCELLED`	—	terminal				

Legacy source states (never written by canonical code; kept so old rows can move forward): `ESTIMATE_APPROVED → REPAIR_PENDING`, `WORK_COMPLETED → READY_FOR_INVOICE`, `INVOICED/PAYMENT_PENDING/PAYMENT_COMPLETED → READY_FOR_DELIVERY`. *Workflow projection* (`resolveCompleted`): see §12 — `APPROVAL` is completed only from `REPAIR_IN_PROGRESS` onward.

9.2 Estimate

State	How reached	Written by
`DRAFT`	created empty by `EstimateServiceImpl.createEstimate(jobCardNumber)`	`EstimateServiceImpl`
`WAITING_FOR_APPROVAL`	estimate totals recalculated after items are added	`EstimateItemServiceImpl`
`APPROVED`	customer `PUT /estimates/{id}/approve` (or manager path `POST /workflow/{n}/estimate/approve`, UI disconnected)	`EstimateServiceImpl.approveEstimateCanonical`
`REJECTED`	customer `PUT /estimates/{id}/reject`	`EstimateServiceImpl.rejectEstimate`
`SENT_FOR_APPROVAL`, `EXPIRED`	*Declared-only*	—

9.3 RepairTask

State	Trigger	Actor	Notes
`PENDING`	created on customer approval (`RepairTaskServiceImpl.createRepairTasks`)	system	one per authorised estimate scope
`ASSIGNED`	`POST /job-assignments` (`JobAssignmentServiceImpl.assignJob`)	manager/SA/owner	
`IN_PROGRESS`	`PUT /job-assignments/{id}/start` or `PUT /repair-tasks/{id}/start`	assigned technician	**refused unless the JobCard is** `REPAIR_IN_PROGRESS`
`COMPLETED`	`PUT /job-assignments/{id}/complete` or `PUT /repair-tasks/{id}/complete`	assigned technician	requires JobCard `REPAIR_IN_PROGRESS`; when all done the job moves to `REPAIR_COMPLETED`
`ON_HOLD`, `CANCELLED`	*Declared-only*		

9.4 JobAssignment

`ASSIGNED → ACCEPTED (accept) → IN_PROGRESS (start) → COMPLETED (complete)`; `CANCELLED` via reassignment. `ON_HOLD`, `QC_PENDING`, `QC_FAILED`, `REWORK` are *Declared-only*. `startJob` requires `ACCEPTED` and (technician type) task `ASSIGNED` + job `REPAIR_IN_PROGRESS`, and **does not touch the JobCard**. `JobAssignmentType.DRIVER` is *Legacy* (§34).

9.5 Booking

`REQUESTED → CONFIRMED` (`confirmBooking`, creates the PICKUP request when pickup was asked) · `REQUESTED → REJECTED` · `REQUESTED/CONFIRMED → CANCELLED` (customer) · `CONFIRMED → COMPLETED` **only** when a Job Card is created with that `bookingId`

(`JobCardServiceImpl.createJobCard`). Trip completion does not change the booking.

9.6 NavigationRequest

State	Written when	By
`REQUESTED`	request created (booking confirm for PICKUP; `POST /navigation/requests` for DELIVERY)	`NavigationRequestServiceImpl.createRequest`
`ASSIGNED`	driver assigned	`NavigationTripServiceImpl.assignDriver`
`IN_PROGRESS`	`startTrip`	`NavigationTripServiceImpl.setRequestStatus`
`COMPLETED`	`completeTrip`	`NavigationTripServiceImpl.setRequestStatus`
`CANCELLED`	*Declared-only* (no cancel workflow)	—

9.7 NavigationTrip

`ASSIGNED` (assignDriver) → `ACCEPTED` (`/accept`) → `IN_PROGRESS` (`/start`) → `COMPLETED` (`/complete`). Sub-state: `arrivedAt` (`/arrive`), `currentLeg` `GARAGE_TO_CUSTOMER → CUSTOMER_TO_GARAGE` (pickup only, `/continue`, which also resets `arrivedAt`). `CANCELLED` is *Declared-only*.

9.8 VehicleHandover

`PENDING` (customer generates code) → `VERIFIED` (driver verifies) · `PENDING → EXPIRED` (10 min, 5 failed attempts, or superseded by a new code). `CANCELLED` is *Declared-only*.

9.9 Delivery

Entity states declared: `PENDING`, `READY_FOR_DELIVERY`, `DELIVERED`. **Only `DELIVERED` is ever written** (`DeliveryServiceImpl.createDelivery` / `completeDeliveryFromTrip`). One `Delivery` row per JobCard and per Invoice (unique).

10. JobCard vs RepairTask

JobCard = the overall vehicle/service lifecycle. RepairTask = one unit of repair work. RepairTask state must never be substituted for JobCard state.

- Creation: RepairTasks appear when the customer approves (`createRepairTasks`, inside the approval transaction).
- Assignment: `JobAssignment` links a technician to a task (`assignJob`); RepairTask `PENDING → ASSIGNED`.
- Technician start/complete: only after the manager has moved the JobCard to `REPAIR_IN_PROGRESS`; both `JobAssignmentServiceImpl.startJob` and `RepairTaskServiceImpl.startRepair` throw *"A manager must proceed this Job Card to repair before a Repair Task can be started."* otherwise. Completing a task requires the job to be `REPAIR_IN_PROGRESS`.
- The only task→job coupling: when **every** task of the job is `COMPLETED`, the job becomes `REPAIR_COMPLETED` (validated). Task start never changes the job.
- QC failure sends the job back to `REPAIR_PENDING`; the manager must confirm again.

11. Customer Approval vs Manager Confirmation

Customer approves does NOT equal manager confirmation.

```
Customer approves → Estimate APPROVED → RepairTasks created → JobCard REPAIR_PENDING
        (Approval step still PENDING in the workflow projection)
Manager taps "Proceed to Repair" → POST /workflow/{n}/repair/start → JobCardServiceImpl.startRepair
        → JobCard REPAIR_IN_PROGRESS → Approval step COMPLETED → Repair is the next step
```

- Authoritative gate: `JobCardServiceImpl.startRepair` — `authorizeProceedToRepair` (MANAGER), estimate must be `APPROVED`, `JobCardStatusValidator` (`REPAIR_PENDING → REPAIR_IN_PROGRESS`); already-`REPAIR_IN_PROGRESS` returns the current state (idempotent).

- Entry points that converge on it: `POST /workflow/{n}/repair/start`, `POST /jobcards/{n}/repair/start`, Job Card screen action `start_repair` (hidden from non-managers by `JobCardProjectionServiceImpl`).
- The pending state is *not* a new JobCard status: it is `REPAIR_PENDING` interpreted by the projection (ADR-004).

12. Guided Workflow Engine

Flutter: `WorkflowShellScreen` (`_steps()`: search, job, inspection, completed [SA only], estimate, estimateItems, estimateSummary, approval, repair, quality, invoice, payment, delivery — 12 visible for a manager), `WorkflowStepDefinition(id, roles, screenBuilder)`, `WorkflowController`, `WorkflowState`, `WorkflowService`.

Backend projection: `GET /workflow/{n}/status` → `WorkflowStatusResponse{jobCardNumber, status, nextStep, progress, completedSteps}` from `ServiceWorkflowServiceImpl.getWorkflowStatus` (`resolveCompleted`, `resolveStep`, `resolveProgress`). `completedSteps` per job status (each row also contains everything above it):

JobCard status	completedSteps adds
any	`CUSTOMER`, `VEHICLE`, `JOB_CARD`
`INSPECTION_COMPLETED`, `ESTIMATE_PENDING`	`INSPECTION`
`WAITING_FOR_APPROVAL`	`ESTIMATE`, `ESTIMATE_ITEMS`, `ESTIMATE_SUMMARY`
`REPAIR_PENDING`, `ESTIMATE_APPROVED`	*(nothing more —* `APPROVAL` *still pending)*
`REPAIR_IN_PROGRESS`	`APPROVAL`
`REPAIR_COMPLETED`	*(same;* `REPAIR` *intentionally not complete — manager reviews then proceeds to QC)*
`QUALITY_CHECK`	`REPAIR`
`READY_FOR_INVOICE`	`QUALITY_CHECK`
`INVOICE_GENERATED`…	`INVOICE`
`PAYMENT_COMPLETED`, `READY_FOR_DELIVERY`	`PAYMENT`
`DELIVERED`, `CLOSED`	`DELIVERY`

`nextStep` uses the shell numbering (Approval 8, Repair 9, QC 10, Invoice 11, Payment 12, Delivery 13) and is informational; Flutter does **not** use it.

Position derivation: `WorkflowController._resolveCurrentStepFromCompletedSteps` picks the first *visible* step whose backend id (`_backendStepIdByLocalId`) is not in `completedSteps` (`search` is skipped once a job exists). It re-runs whenever `workflowStatus` changes (`_onWorkflowStateChanged`) or on `initializeFromWorkflowStatus()`. **Rules for screens:** after a mutation that calls `refreshWorkflowStatus()`, do not call `nextStep()` (it double-advances) — call `initializeFromWorkflowStatus()` (see `EstimateSummaryScreen`, `ApprovalScreen`). Prefer `resumeWorkflow` when other actors (customer) may have changed data. **Resume:** `WorkflowService.resumeWorkflow(n)` → `GET /workflow/{n}/resume` → `syncWorkflowState` (customer, vehicle, job, complaints, inspections, estimate + items, repair tasks, invoice). Entry points: "+ Workflow" FAB (`WorkflowState.reset()`, no API), Job Card "Resume Workflow"/navigate actions (argument = job number), **Booking → Create Job Card** (`WorkflowStartAtJobStep`: customer/vehicle/`bookingId` pre-loaded, starts on the Job Card step via `WorkflowController.goToStepById('job')`). **Job Card detail screen** uses server-driven actions (`JobCardProjectionServiceImpl.allowedActions` → `JobCardActionService`); its actions call the same backend operations as the shell. *Navigate*-kind actions open the shell. **Retained quirks (CLAUDE.md §11):** `WorkflowState.reset()` does not clear `workflowStatus/complaints/repairTasks/assignments`; `closeJob()`/`assignTechnician()` don't refresh status; the employee "Approve Estimate" button is disconnected. `WorkflowState.bookingId` (new) **is** cleared by `reset()`.

13. Backend Service Ownership

Business operation	Controller	Service	Authoritative method	Entity	State written
Job creation	`ServiceWorkflowController.createJob` (`POST /workflow/job`)	`ServiceWorkflowServiceImpl` → `JobCardServiceImpl`	`createJobCard`	`JobCard`, `Complaint`	`OPEN`; Booking `COMPLETED`; pickup request `jobCardId`

Business operation	Controller	Service	Authoritative method	Entity	State written
Inspection start	`POST /workflow/{n}/inspection/start`	`ServiceWorkflowServiceImpl.startInspection`	`InspectionService.startInspection` + `JobCardServiceImpl.startInspection`	`Inspection`, `JobCard`	`INSPECTION_PENDING`
Inspection completion	`POST /workflow/{n}/inspection/complete`	`ServiceWorkflowServiceImpl.completeInspection(list)` (`@Transactional`)	`InspectionService.completeInspection` + `JobCardServiceImpl.completeInspection`	`Inspection`, `JobCard`	`INSPECTION_COMPLETED`
Estimate creation	`POST /estimates` (+ `POST /workflow/{n}/estimate` for job state)	`EstimateServiceImpl.createEstimate`; `JobCardServiceImpl.prepareEstimate`	same	`Estimate` (DRAFT)	`ESTIMATE_PENDING`
Estimate finish	`PUT /estimates/{id}`	`EstimateServiceImpl`	`updateEstimate`	`Estimate`, `JobCard`	`WAITING_FOR_APPROVAL`
Customer approval	`PUT /estimates/{id}/approve`	`EstimateServiceImpl`	`approveEstimate(Long)` → `approveEstimateCanonical`	`Estimate`, `RepairTask`, `JobCard`	Estimate `APPROVED`, tasks `PENDING`, job `REPAIR_PENDING`
Manager repair confirmation	`POST /workflow/{n}/repair/start`, `POST /jobcards/{n}/repair/start`	`JobCardServiceImpl`	`startRepair`	`JobCard`	`REPAIR_IN_PROGRESS`
Repair task start	`PUT /repair-tasks/{id}/start`, `PUT /job-assignments/{id}/start`	`RepairTaskServiceImpl`, `JobAssignmentServiceImpl`	`startRepair`, `startJob`	`RepairTask`, `JobAssignment`	task/assignment `IN_PROGRESS` (never the job)
Repair task completion	`PUT /repair-tasks/{id}/complete`, `PUT /job-assignments/{id}/complete`	same	`completeRepair`, `completeJob`	same + `JobCard`	task `COMPLETED`; job `REPAIR_COMPLETED` when all done
QC	`POST /workflow/{n}/quality-check`; `/jobcards/{n}/quality-check[/pass|/fail]`	`ServiceWorkflowServiceImpl.performQualityCheck`; `QualityCheckServiceImpl`	`passQualityCheck`, `failQualityCheck`	`QualityCheck`, `JobCard`	`QUALITY_CHECK` → `READY_FOR_INVOICE` / `REPAIR_PENDING`
Invoice generation	`POST /workflow/{n}/invoice`	`InvoiceServiceImpl`	`generateInvoice`	`Invoice`, `JobCard`	`INVOICE_GENERATED`
Payment	`POST /workflow/{n}/payment`	`InvoiceServiceImpl`	`receivePayment`	`Invoice`, `JobCard`	invoice `PAID`; job `READY_FOR_DELIVERY`
Pickup request	`POST /bookings/{id}/confirm`	`BookingServiceImpl` → `NavigationRequestService`	`confirmBooking`	`Booking`, `NavigationRequest`	Booking `CONFIRMED`; request `REQUESTED`
Pickup trip	`POST /navigation/trips` + `/accept /start /arrive /continue /complete`	`NavigationTripServiceImpl`	`assignDriver` … `completeTrip`	`NavigationTrip`, `NavigationRequest`	§9.6–9.7
Pickup handover	`GET/POST /navigation/trips/{id}/handover/*`	`HandoverServiceImpl`	`getOrCreateActiveCode`, `verify`	`VehicleHandover`	`PENDING`, `VERIFIED`

Business operation	Controller	Service	Authoritative method	Entity	State written
Pickup completion	`POST /navigation/trips/{id}/complete`	`NavigationTripServiceImpl.completeTrip`	same	trip, request	trip/request `COMPLETED`
Delivery request	`POST /navigation/requests?customerId=`	`NavigationRequestServiceImpl`	`createRequest` → `validateDeliveryRequest`	`NavigationRequest`	`REQUESTED` + `jobCardId`, address, coordinates
Delivery trip	`POST /navigation/trips` …	`NavigationTripServiceImpl`	as pickup	trip	as pickup (no return leg)
Delivery handover	`…/handover/*`	`HandoverServiceImpl`	as pickup, `direction=DELIVERY`	`VehicleHandover`	
Delivery completion	`POST /navigation/trips/{id}/complete`	`NavigationTripServiceImpl.completeTrip` → `DeliveryServiceImpl.completeDeliveryFromTrip`	same	`Delivery`, `JobCard`, trip, request	Delivery `DELIVERED`, job `DELIVERED`
Job delivery (manual)	`POST /deliveries`	`DeliveryServiceImpl`	`createDelivery`	`Delivery`, `JobCard`	same, only when no delivery request exists
Job closure	`PUT /jobcards/{n}/close`, `POST /workflow/{n}/close`	`JobCardServiceImpl`	`closeJobCard`	`JobCard`	`CLOSED`

14. Workflow Coupling Rules

Connection	Who calls whom	Trigger	State change	Auto/explicit	Transactional	Reversible
Job create → inspection start	Flutter `WorkflowService.createJob` → two REST calls	job created	`OPEN` → `INSPECTION_PENDING`	client-chained	**two** transactions (the create, then the start)	no
Inspection → estimate	Flutter `createEstimate` → `/inspection/complete` then `/estimate`	user taps Create Estimate	→ `INSPECTION_COMPLETED` → `ESTIMATE_PENDING`	client-chained	inspection+job atomic (`completeInspection`); estimate prep separate	no
Estimate → customer	`EstimateServiceImpl.updateEstimate`	manager finish	→ `WAITING_FOR_APPROVAL`	explicit	yes	no
Customer approval → repair pending	`EstimateServiceImpl.approveEstimateCanonical` → `RepairTaskServiceImpl.createRepairTasks`, `JobCardRepository`	customer approve	Estimate `APPROVED`, tasks, job `REPAIR_PENDING`	explicit (customer)	**one transaction**	no (no un-approve)
Manager confirm → repair	`JobCardServiceImpl.startRepair`	manager	→ `REPAIR_IN_PROGRESS`	**explicit manager**	yes	no
Task start/complete → job	`JobAssignmentServiceImpl` / `RepairTaskServiceImpl`	technician	task state; job `REPAIR_COMPLETED` on last task	explicit (task) / **automatic** (aggregate)	yes	no
QC fail → rework	`QualityCheckServiceImpl`	QC fail	job `REPAIR_PENDING` (manager must confirm again)	explicit	yes	yes (loop)

Connection	Who calls whom	Trigger	State change	Auto/explicit	Transactional	Reversible
Payment → deliverable	`InvoiceServiceImpl.receivePayment`	payment	job `READY_FOR_DELIVERY`	explicit	yes	no
Booking confirm → pickup request	`BookingServiceImpl.confirmBooking` → `NavigationRequestService`	staff confirm	request `REQUESTED`	**automatic** on confirm when pickup requested	yes	via cancel (no request cancel)
Booking → job	`JobCardServiceImpl.createJobCard` (`resolveBooking`)	job created with `bookingId`	Booking `COMPLETED`; request `jobCardId`	explicit	yes	no
Driver assignment → trip	`NavigationTripServiceImpl.assignDriver`	staff	trip `ASSIGNED`; request `ASSIGNED`	explicit	yes (+ DB unique index backstop)	no
Trip start/complete → request	`NavigationTripServiceImpl.setRequestStatus`	driver	request `IN_PROGRESS` / `COMPLETED`	**automatic**	same transaction as the trip	no
Delivery trip complete → Delivery + job	`NavigationTripServiceImpl.completeTrip` → `DeliveryService.completeDeliveryFromTrip`	driver, after evidence + OTP	Delivery row, job `DELIVERED`	**automatic**	**one transaction** with the trip completion	no
Trip ↔ Workflow UI	none (Delivery screen buttons only)	—	—	—	—	—
Journey ← all	`CustomerVehicleJourneyServiceImpl` (read-only)	customer opens journey	none	derived	read-only	n/a

15. Booking Architecture

`BookingController` (`/api/v1/bookings`): customer `POST`, staff `POST /phone` (operational roles), `GET /my`, `GET /garage`, `GET /{id}`, `POST /{id}/confirm|reject|cancel`. `BookingSource`: `CUSTOMER_APP`, `PHONE`. Confirm with `pickupRequested` builds a PICKUP `NavigationRequest` (address + coordinates copied) and stores `navigationRequestId`. **Booking → Job Card:** `BookingDetailScreen` shows **Create Job Card** for staff when `canCreateJobCardFromBooking` (status `CONFIRMED` and either no pickup or its trip is `COMPLETED`; UI gate only). It preloads customer (`CustomerService.getById`) and vehicle (`VehicleRepository.getById`) into `WorkflowState`, sets `bookingId`, and pushes `Routes.employeeWorkflow` with `WorkflowStartAtJobStep`. `JobStepScreen` sends `CreateJobCardRequest.bookingId`. Backend `JobCardServiceImpl.resolveBooking` requires: booking exists, same garage (else not-found), `CONFIRMED`, same vehicle, customer matches vehicle owner, no other job already uses it. Then it sets `JobCard.bookingId`, Booking `COMPLETED`, and `NavigationRequest.jobCardId` of the booking's pickup request. Walk-in jobs omit `bookingId` and are unchanged.

16. Pickup Architecture (deep)

```
Booking(REQUESTED) —confirm—▶ Booking(CONFIRMED) + NavigationRequest(PICKUP, REQUESTED)
 — staff "Assign Driver" (AssignDriverSheet → POST /navigation/trips) —▶ Trip ASSIGNED, request ASSIGNED
 — driver Accept —▶ Trip ACCEPTED — Start —▶ Trip IN_PROGRESS (leg GARAGE_TO_CUSTOMER), request IN_PROGRESS
 — GPS stream + route to customer — "I Have Arrived" —▶ arrivedAt set
 — 5 BEFORE_PICKUP photos (FRONT, REAR, LEFT, RIGHT, ODOMETER) — customer generates OTP — driver verifies —▶ handover VERIFIED
 — "Start Return to Garage" —▶ POST /continue: leg CUSTOMER_TO_GARAGE, arrivedAt=null, destinationAddress="GARAGE"
 — GPS + route to garage — "I Have Arrived" —▶ arrivedAt set
 — "Complete Pickup Trip" —▶ POST /complete: Trip COMPLETED, request COMPLETED
 — staff Create Job Card from the booking (job linked, booking COMPLETED, request.jobCardId set)
```

- **Completion preconditions** (`NavigationTripServiceImpl.completeTrip`, pickup): trip `IN_PROGRESS`; leg `CUSTOMER_TO_GARAGE` ("Pickup trip must return to garage before completion."); `arrivedAt != null` ("Mark arrival at the garage before completing the

pickup trip."); all required `BEFORE_PICKUP` photos; verified PICKUP handover (re-checked defensively).

- **No garage-side OTP, by design:** custody transfer is confirmed once, at the customer (OTP + photos) before the return leg. At the garage there is no counter-party in the app to generate a code; the driver's arrival plus the earlier verified handover is the completion condition. `HandoverServiceImpl` therefore *refuses* code generation/verification for a pickup trip already on the return leg.
- **Return-leg destination detail:** the backend sets `destinationAddress` to the literal `"GARAGE"` and `NavigationTripResponse.destinationLatitude/Longitude` keep pointing at the customer's pickup coordinates. `ActiveTripScreen` resolves the real garage via `GET /garages/{id}` (`GarageService`, user's own `garageId`) and uses it for return-leg arrival/route. Anything else consuming a trip's destination on the return leg must do the same.
- **Driver UI:** `DriverDashboard._TripCard` renders `NavigationTrip.nextDriverAction` (`accept`, `start`, `arrive`, `evidenceAndHandover`, `completePickup`, `none`); `isPickupReturnLeg` = `tripType=='PICKUP' && currentLeg=='CUSTOMER_TO_GARAGE'`. `ActiveTripScreen` hides "View Confirmation Code" on the return leg.
- **Customer journey:** `DRIVER_ASSIGNED → DRIVER_EN_ROUTE → DRIVER_ARRIVED → EN_ROUTE_TO_GARAGE`; once the booking's job exists (request linked to job) the job-card stage takes over (§22).

17. Delivery Architecture (deep)

```
JobCard READY_FOR_DELIVERY
 → Delivery step (DeliveryScreen) restores state: GET /navigation/requests/job-card/{id}/delivery (404 = none)
 → "Request Delivery & Assign Driver": CreateDeliveryRequestSheet → LocationPickerScreen (map) → POST
/navigation/requests?customerId=
      NavigationRequest(DELIVERY, jobCardId, deliveryAddress, deliveryLatitude, deliveryLongitude, REQUESTED)
 → AssignDriverSheet → POST /navigation/trips → Trip ASSIGNED, request ASSIGNED
 → driver Accept → Start → GPS/route to delivery coordinates → Arrive → 5 DELIVERY photos → customer OTP → driver
verifies
 → POST /{trip}/complete:
      validateRequiredMedia(DELIVERY) + requireVerifiedHandover(DELIVERY)
      Trip COMPLETED; request COMPLETED
      DeliveryServiceImpl.completeDeliveryFromTrip(request.jobCardId, driverName, "Customer (OTP verified)")
          → Delivery(DELIVERED, deliveredBy, receivedBy) ; JobCard READY_FOR_DELIVERY → DELIVERED   (same transaction)
 → manager Close Job → CLOSED
```

- **Request rules** (`NavigationRequestServiceImpl.validateDeliveryRequest`): `jobCardId` required; address **and** both coordinates required; job in caller's garage and the request's garage; job `READY_FOR_DELIVERY`; request vehicle/customer must match the job's; no other non-cancelled DELIVERY request for the job. DB backstop: partial unique index (V61).
- `completeDeliveryFromTrip` **rules:** garage match; no existing Delivery; invoice found via `InvoiceRepository.findByEstimateJobCardId`; invoice `GENERATED` and `PAID`; then create Delivery and `JobCardStatusValidator` + set `DELIVERED`. Any failure rolls back the trip completion (same `@Transactional` boundary).
- **Why manual** `POST /deliveries` **still exists:** a customer who collects the car at the garage has no delivery trip. It is now **rejected** when any DELIVERY request exists for the job ("A delivery trip exists for this Job Card…") and when the invoice belongs to another job. The Delivery screen hides "Mark Vehicle Delivered (collected at garage)" once a request exists.
- **Delivery step UI states:** loading → *no request* (button "Request Delivery & Assign Driver") → `REQUESTED` ("Assign Delivery Driver") → `ASSIGNED/IN_PROGRESS` ("Track Delivery") → `COMPLETED` ("View Delivery"; job is `DELIVERED`, "Close Job" enabled). Returning from `ActiveTripScreen` calls `_refreshDelivery` (reload request + `resumeWorkflow`).
- **Legacy:** delivery requests created before V61 have `job_card_id = NULL`; their trip completes but touches no job.
- **Not implemented:** automatic driver selection, workload balancing, driver rejection, reassignment, delivery/trip cancellation.

18. Delivery Location

`CreateDeliveryRequestSheet` (bottom sheet, `show(context, customerId, vehicleId, garageId, jobCardId)`) offers **Select location on map** (pushes the existing `LocationPickerScreen` with `LocationPickerMode.delivery` and a `LocationSearchService`) and the older **current device location** shortcut (`PickupLocationProvider`). The picker returns a `PickupLocation{latitude, longitude, address?}`; the sheet fills coordinates and an editable address field. `resolveDeliveryAddress` keeps the address if present, otherwise `"<lat>, <lng>"` (6 decimals). Submit requires coordinates. `NavigationTripService.createDeliveryRequest` sends `requestType=DELIVERY, jobCardId, deliveryAddress, deliveryLatitude, deliveryLongitude, scheduledAt(now+2 min)`. Persistence: `navigation_request.delivery_address/latitude/longitude`; consumed by

`NavigationTripServiceImpl.toResponse` as the trip destination for DELIVERY trips → `NavigationTrip.destinationLatitude/Longitude` → `ActiveTripScreen` / `LiveTripState` route.

19. Navigation Architecture

- **Driver GPS:** `DriverLocationTracker.start` — `Geolocator.getPositionStream`, high accuracy, `distanceFilter: 10` **m**, Android foreground-service notification. Started by `ActiveTripScreen._maybeStartDriverGps` for the driver; runs only while that screen is open.
- **Publish:** `LocationSocketService.publishLocation` → STOMP `/app/location/update` → `LocationWebSocketController` → `DriverLocationServiceImpl` (saves `DriverCurrentLocation`, appends `DriverLocationHistory` (`driver_location_history`), broadcasts via `SimpMessagingTemplate`).
- **Subscribe:** `/topic/trips/{tripId}/location` (customer, manager, driver), guarded by `TripLocationTopicInterceptor`. Socket: endpoint `/ws`, `reconnectDelay` 5 s, connection timeout 10 s, topics re-subscribed after reconnect.
- **REST fallback:** `LiveTripState` polls `GET /navigation/trips/{id}` every **10 s** (`_statusPollInterval`) and reads `GET /navigation/trips/{id}/location` when no recent socket fix.
- **Route:** `RouteService` → `POST /navigation/routes` (`RouteRequest{origin,destination}`) → `NavigationRouteServiceImpl`: Google Routes when configured (`GOOGLE_MAPS_ROUTES_API_KEY`), else OSRM. Recomputed only after the driver moved ≥ **150 m** since the last route (`_routeRecalculationDistanceMeters`).
- **Arrival:** the backend gate is the driver's `POST /arrive`; a 10 m GPS test (`kArrivalRadiusMeters`) drives only display labels.
- **Destination selection:** pickup outbound = request pickup coordinates; delivery = request delivery coordinates; pickup return leg = garage from `/garages/{id}` (§16). No coordinates → map shows the driver but no destination pin/route (`hasDestinationCoordinates`).
- **Screens:** `ActiveTripScreen` (driver/customer/manager), `ManagerFleetMapScreen` (`GET /navigation/trips/garage/{id}/fleet`), `DriverTripHistoryScreen`. Maps: `google_maps_flutter` on Android, `flutter_map` elsewhere.

20. OTP Architecture

`HandoverServiceImpl` (`VehicleHandover`, `V38`, `V39` = at most one pending per trip):

Aspect	Behaviour
Generation	**customer** `GET /navigation/trips/{tripId}/handover/code` (`getOrCreateActiveCode`); allowed only if the caller owns the request, `trip.arrivedAt != null`, trip `IN_PROGRESS`, and the trip is not a pickup already on its return leg
Code	6 digits, `SecureRandom`; only a `PasswordEncoder` hash (`code_hash`) is stored; audit events never include the code
Delivery to the driver	shown in the customer app, read out to the driver — not sent by SMS/e-mail
Expiry	10 minutes (`EXPIRY_MINUTES`); requesting a new code marks the old pending one `EXPIRED`
Verification	**driver** `POST …/handover/verify {code}` (`VerifyHandoverRequest.code`); trip must be the driver's own and in the driver's garage; needs a `PENDING`, unexpired code
Attempts	`MAX_FAILED_ATTEMPTS = 5`; wrong code increments; at 5 the code is `EXPIRED` and a new one is needed
Direction	`direction = trip.tripType` (PICKUP or DELIVERY) — identical flow
Effects	`VERIFIED` + `verifiedAt/verifiedByDriverId`; audit `OTP_VERIFIED` and custody event. The trip does **not** move by itself: the driver presses "Start Return to Garage" (`continue`, pickup) or "Complete Delivery" (`complete`, delivery) in `HandoverVerifyScreen._advance`
Gate	`continueTrip` (pickup) and `completeTrip` (delivery; and pickup, re-checked) call `requireVerifiedHandover`
Status	`GET …/handover/status` (garage staff) returns latest handover

21. Trip ↔ Booking ↔ JobCard Relationships

```
Booking ──navigationRequestId──▶ NavigationRequest(PICKUP) ──unique──▶ NavigationTrip ──▶ VehicleHandover
   │                                  │ jobCardId (set when the job is created from the booking)   └▶
NavigationTripMedia
   └── JobCard.bookingId ──▶ JobCard ◀──┘
                            ├─ Estimate ─ EstimateItem ─▶ RepairTask ─ JobAssignment
                            ├─ Invoice(estimate) ─▶ Delivery(jobCard 1:1, invoice 1:1)
                            └─ NavigationRequest(DELIVERY).jobCardId ──unique per live request──▶ NavigationTrip
```

The only reference added by the latest change is `navigation_request.job_card_id` (V61). Trips still have no direct FK to a JobCard; they reach it through their request. Vehicle-based heuristics remain only as a fallback in the customer journey.

22. Customer Journey Logic

`CustomerVehicleJourneyServiceImpl.getActiveJourneys()` (customer portal `GET /customer/vehicle-journeys/active`, screen `LiveVehicleJourneyScreen`). Per vehicle of the authenticated customer it batch-loads: latest **CONFIRMED booking**, latest **REQUESTED navigation request**, latest **ASSIGNED/ACCEPTED/IN_PROGRESS trip**, latest **non-**`CLOSED/CANCELLED` **job card** (note `DELIVERED` is still "active" and shows `COMPLETED`). Selection (`deriveStage`):

1. active trip → `deriveFromTrip` (`DRIVER_ASSIGNED`, `DRIVER_EN_ROUTE`, `DRIVER_ARRIVED`, `EN_ROUTE_TO_GARAGE` for pickup; `HANDOVER` for delivery);
2. else job card → `deriveFromJobCard` (`ARRIVED_AT_GARAGE`, `INSPECTION`, `ESTIMATE_PENDING`, `CUSTOMER_APPROVAL_REQUIRED`, `REPAIR_PENDING`, `REPAIR_IN_PROGRESS`, `QUALITY_CHECK`, `PAYMENT_PENDING`, `VEHICLE_READY`, `COMPLETED`);
3. else pending request → `BOOKING_CONFIRMED` ("arranging your pickup driver");
4. else confirmed booking → `BOOKING_CONFIRMED`.

Direct-link rule (new): if the active trip is a PICKUP whose `NavigationRequest.jobCardId` equals the vehicle's job card, the trip is ignored for stage selection (the vehicle is at the garage; the job card is the truth). A DELIVERY trip keeps priority (the customer must see the handover). Trips without the link (legacy, or before the job is created) keep the old "trip wins" behaviour. **Edge cases:** a pickup trip left uncompleted and not yet linked still shows `EN_ROUTE_TO_GARAGE`; the vehicle-based selection can mis-pair records when a vehicle has overlapping active records; a stuck trip never auto-expires (no cancellation workflow).

23. Media Architecture

Direct-to-R2, private bucket, presigned URLs. Spring never proxies media bytes; the old multipart limit was not raised.

New image

`MediaPickerSheet` (image_picker, optimized at pick) → capture metadata taken **at pick time** (`capturedAt`, one-shot GPS + reverse-geocoded address, sent with the offset via `isoWithLocalOffset`) → `MediaUploadQueueService.enqueue` (persistent queue) → `POST /job-cards/{id}/media/upload-intent` (`MediaServiceImpl.createUploadIntent`: row `evidence_only=true`, `captured_by` from the authenticated user, storage key `…_source.jpg`) → direct PUT → `POST /media/{id}/complete` → `MediaProcessingScheduler` (15 s poll) renders `…_evidence.jpg` (full resolution + footer: stage, captured time, captured-by, address, `Lat/Lng`, accuracy, GarageST) and `…_thumbnail.jpg` (320 px longest edge, from the evidence), sets `storage_key = evidence_key`, saves `COMPLETED`, **then** deletes the `_source` object. Retry: failures keep the row `UPLOADED`, `retry_count++`, `FAILED` after 5 (source kept); a lost save after rendering is detected and not re-rendered.

New video

Pick/record → `MediaOptimizationService.renderEvidenceVideo` → Android `EvidenceVideoProcessor` (Media3 Transformer, **single pass**): reads rotation and computes the target in the video's own orientation (long edge ≤ 1920, short edge ≤ 1080, even dimensions, never squeezed into a landscape frame), bitrate = 80 % of the source (`qualityPercent = 80`, H.264/AAC MP4), draws the footer as a full-frame bitmap overlay, then extracts a JPEG (≤ 480 px longest edge) from the **finished** file → upload intent (returns two presigned URLs: `…_evidence.mp4`, `…_thumbnail.jpg`) → PUT both → `/complete {thumbnailUploaded:true, durationSeconds}` (backend verifies the thumbnail object exists). Evidence text is built on-device from persisted queue-item metadata (`buildEvidenceLines`) and mirrors `MediaEvidenceText` on the backend (same literals asserted in both test suites). Local outputs live in

`<temp>/media_optimized/{localId}.mp4|.jpg`, are re-used on retry (never re-rendered once the paths are persisted), deleted after completion, and orphans are swept at app start. A crashed render restarts from the untouched original.

Keys, access, cache, TTL

- Key convention: `GarageST/{garageCode}/{jobCardNumber}/{stage}/{jobCardNumber}_{stage}_{seq}_{source|evidence|thumbnail}.{ext}` (`MediaKeys.derive`); sequence from `getNextSequence`; no per-media folders.
- Access: `GET /media/{id}/access?variant=evidence|thumbnail` (employee) and `GET /customer/jobcards/{n}/media/{id}/access` (customer). `evidence_only` rows always resolve to the evidence (also for a stale `original`), and an image still being processed is `available=false` — the staging source is never signed.
- TTL: upload URL `R2_UPLOAD_URL_TTL_MINUTES` (default 15), playback `R2_PLAYBACK_URL_TTL_MINUTES` (default 10).
- Flutter cache keys: `media-video-{mediaId}` (stable, never the URL), `media-evidence-{id}`, `media-thumb-{id}`.
- Failure model: `MEDIA_STORAGE_NOT_CONFIGURED` is non-retryable; network/5xx retry with backoff (30 s, 2 m, 5 m, 15 m, 30 m); an upload URL that expired is re-requested (which creates a new row — known pre-existing behaviour).
- **Legacy compatibility:** Google Drive rows (proxied `/content`), and R2 rows with original + thumbnail/evidence keep working; the `evidence` variant falls back to the stored original when no evidence exists. Nothing was migrated or deleted. R2 delete removes a row's storage, evidence and thumbnail objects.
- **There is no separate original asset for NEW media.**

24. Trip Media vs JobCard Media

`JobCardMedia` ≠ `NavigationTripMedia`.

	JobCardMedia	NavigationTripMedia
Owner	a JobCard	a NavigationTrip
Storage	Cloudflare R2 (Drive for legacy)	server local disk (`LocalMediaStorageService`, `garageos.media.storage-path`) — *Config-dependent durability on Render*
Upload	presigned direct PUT + queue	multipart through Spring (`POST /navigation/trips/{id}/media`)
Processing	evidence footer + thumbnail	none
Metadata	captured-by/at, lat/lng, accuracy, address	shot type, stage, lat/lng, capturedBy/At
Purpose	inspection/repair evidence for the job	pickup/delivery condition evidence; required shot set gates trip continue/complete
Intentionally separate; connect only by ids if ever needed. The latest change did not touch either.		

25. Database Architecture

Key entities: `User` (garage, role) · `Garage` · `Customer` · `Vehicle` · `Booking` (customerId, vehicleId, garageId, navigationRequestId) · `JobCard` (garage, vehicle, customer, `bookingId`, status) · `Complaint` · `Inspection` · `Estimate` (jobCard) · `EstimateItem` (selected) · `RepairTask` (jobCard, jobAssignment) · `JobAssignment` (user, repairTask, type) · `QualityCheck` · `Invoice` (estimate 1:1) · `Delivery` (jobCard 1:1, invoice 1:1, deliveredBy, receivedBy) · `NavigationRequest` (customerId, vehicleId, garageId, requestType, pickup/delivery address + coordinates, `jobCardId`, status) · `NavigationTrip` (navigationRequestId unique, driverId, tripType, currentLeg, status, source/destinationAddress, accepted/started/arrived/completedAt) · `VehicleHandover` (tripId, direction, codeHash, status, expiresAt, failedAttempts) · `NavigationTripMedia` · `JobCardMedia` (evidence_only, storage/thumbnail/evidence keys, capture metadata) · `AuditEvent`. `spring.jpa.hibernate.ddl-auto=validate`: an entity field without a migrated column fails startup.

V61 — `V61__add_navigation_request_job_card_reference.sql`

- **Table:** `navigation_request`. **Column:** `job_card_id BIGINT`, **nullable**, no default. **No foreign-key constraint** (the entity stores a plain `Long`; V61 declares none); integrity is enforced in services.

- **Indexes:** `idx_navigation_request_job_card (job_card_id)`; and **partial unique** `idx_navigation_request_one_delivery_per_job ON navigation_request (job_card_id) WHERE request_type = 'DELIVERY' AND status <> 'CANCELLED' AND job_card_id IS NOT NULL`.
- **Why the partial index:** database backstop for "at most one live delivery request per job card", so two concurrent "request delivery" submissions cannot create two deliveries for one job. Cancelled requests and legacy rows (NULL) are excluded; PICKUP rows are excluded entirely.
- **Who requires `job_card_id`:** new DELIVERY requests (service-level, `validateDeliveryRequest`). PICKUP requests have it set later, when the job is created from their booking; otherwise NULL.
- **Legacy rows:** all NULL; they remain valid; their delivery trips complete without a job effect.
- **Rollback:** the column is additive and nullable, so an older backend build runs fine against the new schema (it ignores the column). The reverse is not true (new build on the old schema fails `validate`). To undo the schema, add a **new** migration (V62 is now the notification migration; use the next free number) that drops the index/column — never edit V61 once applied. Dropping the column loses the delivery↔job link.
- **Flyway:** applied automatically at application start (`spring.flyway.enabled=true`, `spring.flyway.locations=classpath:db/migration`) before JPA validation; recorded in `flyway_schema_history`.
- *Other schema changes in this line of work:* `V60` `job_card_media.evidence_only BOOLEAN NOT NULL DEFAULT FALSE`. (`V59`, `V58` are the earlier media migrations.)
- **Not verified here:** V61 was not executed against a PostgreSQL instance in this environment (*Environment required*).

V62 — `V62__create_notification_system.sql` (notifications)

Additive migration (no existing table altered). Applied automatically by Flyway at start-up.

Table	Purpose	Major fields	Constraints / indexes
`device_token`	one row per FCM registration token (an app installation, not an identity)	`user_id` (FK `users`), `token` VARCHAR(512), `platform` (ANDROID/IOS), `app_version`, `is_active`, `last_seen_at`, `created_at`, `updated_at`	**UNIQUE** `uk_device_token_token (token)`; partial `idx_device_token_user_active (user_id) WHERE is_active`
`notification`	in-app history, one row **per recipient**; source of truth	`user_id` (FK), `garage_id`, `type`, `category`, `priority`, `title`, `message`, `entity_type`, `entity_id`, `job_card_id`, `event_key`, `is_read`, `read_at`, timestamps	**UNIQUE** `uk_notification_event_user (event_key, user_id)` (idempotency); `idx_notification_user_created (user_id, created_at DESC, id DESC)`; partial `idx_notification_user_unread (user_id) WHERE is_read = FALSE`; `idx_notification_job_card`; `idx_notification_garage_created`
`notification_outbox`	transactional outbox written in the business transaction	`event_key`, `event_type`, `garage_id`, `entity_type`, `entity_id`, `job_card_id`, `payload` TEXT (id-only facts as JSON), `status` (PENDING/PROCESSING/SENT/FAILED), `attempt_count`, `next_attempt_at`, `locked_at`, `last_error`, `processed_at`	**UNIQUE** `uk_notification_outbox_event_key (event_key)`; `idx_notification_outbox_status_next (status, next_attempt_at)`
`notification_delivery`	per-device push state of one notification	`notification_id` (FK), `device_token_id` (FK), `status` (PENDING/SENT/FAILED/INVALID_TOKEN), `attempt_count`, `next_attempt_at`, `sent_at`, `last_error`	**UNIQUE** `uk_notification_delivery_pair (notification_id, device_token_id)`; `idx_notification_delivery_status_next (status, next_attempt_at)`

- **Relationships:** `notification → users`; `device_token → users`; `notification_delivery → notification, device_token`. `notification.garage_id`, `job_card_id` and the outbox's entity columns are plain ids (no FK), like `navigation_request.job_card_id` (V61).
- **Conventions followed:** BIGSERIAL ids, `TIMESTAMP` (no zone), VARCHAR enums, `TEXT` payload (no JSONB), `ddl-auto=validate`.
- **Idempotency:** `event_key` + `user_id` (see §28A.7). **Lifecycle:** outbox PENDING → PROCESSING → SENT/FAILED; delivery PENDING → SENT/FAILED/INVALID_TOKEN; notification `is_read`.
- **Rollback:** the tables are additive, so an older backend build runs against the V62 schema. To remove them, add a **new** migration (next free number) dropping `notification_delivery`, `notification_outbox`, `notification`, `device_token` in that order — never edit V62 once applied. Dropping loses notification history. Day-to-day rollback is the feature flag (`NOTIFICATIONS_ENABLED=false`), which leaves the tables in place.
- **Verified by:** `NotificationPipelineIT` (Flyway applied V62 to the isolated `garageos_test` database and the native SQL ran against it).

26. Flyway Migrations

Location `src/main/resources/db/migration`, names `V{n}__snake_case_description.sql`, strictly increasing. **Never modify an applied migration.** New columns: nullable or with a default; prefer partial/unique indexes as backstops for service rules. Test on PostgreSQL (partial indexes are PostgreSQL syntax). Recent: V57 account deletion · V58 media provider fields · V59 media audit metadata · V60 media evidence flag · V61 navigation_request job reference · **V62 notification system (device_token, notification, notification_outbox, notification_delivery)**.

27. API Reference (business-critical)

Base `/api/v1`. "Auth" lists the enforced restriction; *svc* = enforced in the service, not by annotation. Errors surface as `ApiException.message` in Flutter.

Workflow — `ServiceWorkflowController` `/workflow`

Method & path	Auth	Purpose / inputs	State change	Notable failures
`POST /job`	authenticated, garage user	create job (`CreateJobCardRequest`: vehicleId, odometer, complaints[], estimatedDeliveryDate, remarks, **bookingId?**)	`OPEN`; Booking `COMPLETED` if bookingId	"User is not associated with a garage."; booking rules (§15)
`POST /{n}/inspection/start`	authenticated (svc garage scope)	creates inspection rows	`INSPECTION_PENDING`	invalid transition
`POST /{n}/inspection/complete`	authenticated	body: one `CreateInspectionRequest` per complaint	`INSPECTION_COMPLETED`	size mismatch
`POST /{n}/estimate`	authenticated	prepare estimate stage	`ESTIMATE_PENDING`	invalid transition
`POST /{n}/estimate/approve`	MANAGER (svc)	employee-side approval (UI disconnected)	as customer approval	"Only a Manager can approve this estimate."
`POST /{n}/repair/start`	**MANAGER (svc)**	manager confirmation	`REPAIR_IN_PROGRESS`	"Only a Manager can proceed a Job Card to repair."; "Estimate must be customer-approved before repair can start."; invalid transition
`POST /{n}/repair/complete`	authenticated	*Legacy* check-only wrapper	none	—
`POST /{n}/quality-check`	M/SA/OWNER	start QC	`QUALITY_CHECK`	invalid transition
`POST /{n}/invoice`, `/invoice/accept`	M/SA/OWNER ; CUSTOMER	generate / accept	`INVOICE_GENERATED`	
`POST /{n}/payment`	M/SA/OWNER/CUSTOMER (customer ownership svc)	receive payment	invoice `PAID`, `READY_FOR_DELIVERY`	"Invoice must be accepted before payment can be received."
`POST /{n}/delivery`	authenticated	`readyForDelivery` (UI unused)	`READY_FOR_DELIVERY`	—
`POST /{n}/close`	M/SA/OWNER	close	`CLOSED`	invalid transition
`GET /{n}/status`	authenticated	`WorkflowStatusResponse`	none	
`GET /{n}/resume`	authenticated	full snapshot for `WorkflowState`	none	

Method & path	Auth	Purpose / inputs	State change	Notable failures
`GET /{n}/repair-tasks`	authenticated	tasks	none	

JobCard — `JobCardController` `/jobcards`

`GET /{id}/view` (role-dependent projection: customer / technician / operational; `allowedActions`, `visibleSections`) · `POST /{n}/inspection/start|complete`, `/estimate/prepare`, `/estimate/approve`, `/repair/start` (same `startRepair`), `/quality-check`, `/quality-check/pass|fail` (M/SA/OWNER), `PUT /{n}/ready-for-delivery`, `PUT /{n}/close` (M/SA/OWNER).

Estimate — `EstimateController` `/estimates`

`PUT /{id}` finish (staff; `ESTIMATE_PENDING → WAITING_FOR_APPROVAL`) · `PUT /{id}/approve` and `/reject` (**CUSTOMER owner only**; not-found otherwise; "Estimate is already approved.").

Repair — `/repair-tasks`, `/job-assignments`

`PUT /repair-tasks/{id}/assign` (M/SA/OWNER) · `PUT /repair-tasks/{id}/start|complete` (assigned technician or operational staff) · `POST /job-assignments` (M/SA/OWNER) · `PUT /job-assignments/{id}/accept|start|complete` (the assignee) · `PUT /job-assignments/{id}/reassign` (M/SA/OWNER). Start/complete guards in §10.

Booking — `/bookings`

`POST /` (customer) · `POST /phone` (M/SA/OWNER) · `GET /my`, `/garage`, `/{id}` · `POST /{id}/confirm|reject|cancel`. Job linkage is via `POST /workflow/job` `bookingId`, not a booking endpoint.

Navigation — `/navigation`

Method & path	Caller	Purpose	Key validation / state
`POST /requests?customerId=`	garage staff	create request (`CreateNavigationRequest`)	DELIVERY: §17 rules; PICKUP created by booking confirm
`GET /requests/{id}`, `/customer/{id}`, `/garage/{id}`, `/{id}/trip`	per svc	lookups	`/trip` restricted to customer/driver/garage staff
`GET /requests/job-card/{jobCardId}/delivery`	garage staff	rediscover a job's delivery request	404 if none / other garage
`POST /trips`	garage staff	assign driver (`{navigationRequestId, driverId}`)	request `REQUESTED`; driver active DRIVER of the garage; unique trip per request → trip `ASSIGNED`, request `ASSIGNED`
`POST /trips/{id}/accept?driverId=`	that driver		trip `ASSIGNED → ACCEPTED`
`POST /trips/{id}/start?driverId=`	that driver		`ACCEPTED → IN_PROGRESS`; request `IN_PROGRESS`
`POST /trips/{id}/arrive?driverId=`	that driver	sets `arrivedAt`	trip `IN_PROGRESS`
`POST /trips/{id}/continue?driverId=`	that driver	pickup outbound → return leg	pickup only; outbound leg; required `BEFORE_PICKUP` photos; verified PICKUP handover
`POST /trips/{id}/complete?driverId=`	that driver	finish	pickup: return leg + arrival + photos + handover; delivery: photos + handover + `completeDeliveryFromTrip`; request `COMPLETED`
`GET /trips/{id}`, `/{id}/location`	customer owner / driver / garage staff (`NavigationTripAccessGuard`)	trip / last location	not-found otherwise
`GET /trips/driver/{id}`, `/driver/{id}/history`, `/garage/{id}/fleet`	driver (self) / garage staff	lists	

Method & path	Caller	Purpose	Key validation / state
`POST /routes`	authenticated	route between two points	Google → OSRM fallback
`POST /trips/{id}/media`, `GET …/media`, `…/media/{stage}`, `…/media/{mediaId}/content`	driver upload; viewers	trip evidence	upload only `IN_PROGRESS` and after arrival

Handover — `/navigation/trips/{tripId}/handover`

`GET /code` (customer owner) · `POST /verify {code}` (driver) · `GET /status` (garage staff) · `POST /evidence-viewed` (customer). Rules §20.

Delivery — `/deliveries`

`POST /` (M/SA/OWNER) `CreateDeliveryRequest{jobCardId, invoiceId, deliveredBy?, receivedBy?, remarks?}` — garage collection only; rejected if a delivery request exists, if the invoice isn't the job's, if invoice not `GENERATED` + `PAID`, or a Delivery already exists. Trip-driven completion has no endpoint of its own: it happens inside `POST /navigation/trips/{id}/complete`. `GET /deliveries/{id}`, `GET /deliveries` are lookups.

Media

`POST /job-cards/{id}/media/upload-intent` (`UploadIntentRequest`: stage, contentType, fileSize, repairTaskId?, capturedAt, latitude, longitude, locationAccuracyMeters, locationName) → `UploadIntentResponse` (mediaId, uploadUrl, requiredHeaders, storageKey, uploadSessionId, **thumbnailUploadUrl/thumbnailRequiredHeaders for VIDEO**) · direct `PUT` to R2 · `POST /media/{id}/complete` (`uploadSessionId`, durationSeconds?, `thumbnailUploaded?`) idempotent · `GET /media/{id}/access?variant=` (employee) · `GET /customer/jobcards/{n}/media/{id}/access` (customer) · `GET /job-cards/{id}/media` · `PUT /job-cards/{id}/media/{mediaId}/visibility` · `DELETE /media/{id}` (privileged) · `POST /job-cards/{id}/media` (*Legacy* multipart). Failure: `MEDIA_STORAGE_NOT_CONFIGURED`.

Notifications — `NotificationController` `/notifications`, `DeviceTokenController` `/notifications/devices` (new 2026-10-02)

All endpoints require authentication (default `anyRequest().authenticated()`; no role restriction) and always act as the **JWT principal**; no user id is accepted from the client. Responses use the standard `ApiResponse` envelope.

Method & path	Auth	Request	Response (`data`)	Purpose
`POST /notifications/devices`	authenticated	`{token (≤512, required), platform? (ANDROID|IOS, default ANDROID), appVersion? (≤50)}`	`{id, platform, active}`	register/refresh the device token; reassigns a token held by another user; reactivates
`POST /notifications/devices/unregister`	authenticated	`{token}`	none	deactivate the caller's token (token in the body, not the URL); ignores tokens owned by others
`DELETE /notifications/devices/{id}`	authenticated	—	none	deactivate by device-token id; only if owned by the caller
`GET /notifications?page=0&size=10`	authenticated	page (default 0), size (default 10, **max 50**)	Spring `Page` of `{id, type, category, priority, title, message, entityType, entityId, jobCardId, garageId, read, readAt, createdAt}`, newest first	the caller's notification history
`GET /notifications/unread-count`	authenticated	—	`{count}`	badge (single indexed count)
`PATCH /notifications/{id}/read`	authenticated	—	none	mark one read; idempotent; **404** if not the caller's

Method & path	Auth	Request	Response (`data`)	Purpose
`PATCH /notifications/read-all`	authenticated	—	none	mark all the caller's notifications read

28. WebSocket Architecture

`WebSocketConfig`: endpoint `/ws` (plain and SockJS, all origins), application prefix `/app`, simple broker `/topic`. Publish `/app/location/update` (`DriverLocationRequest`: driverId, tripId, lat/lng, speed, heading, accuracy). Subscribe `/topic/trips/{tripId}/location`. Authorization per subscription: `TripLocationTopicInterceptor` → `NavigationTripAccessGuard.authorizeViewer`. Client: `LocationSocketService` (singleton), 5 s reconnect, resubscribe on connect; if the socket is down `LiveTripState` keeps working through REST polling. The in-memory simple broker means a single backend instance (no external broker). **Notifications do not use STOMP:** the broker has no per-user destinations and authorises only `/topic/trips/{id}/location`; push is delivered through FCM (§28A).

28A. Push Notification System

Status: IMPLEMENTED · PHYSICALLY VALIDATED — ANDROID (see 28A.15) · CONFIGURATION-DEPENDENT (needs Firebase project files and two backend environment variables). **iOS push: NOT IMPLEMENTED.** WebSocket/STOMP is **not** used for notifications; it stays dedicated to live-trip location (§28).

28A.1 Architecture and flow

```
Business mutation (e.g. receivePayment, failQualityCheck, assignDriver)
        │  same @Transactional method
        ▼
NotificationEvents.publish… → NotificationEventPublisher.publish(event)
        │  INSERT … ON CONFLICT DO NOTHING (caller's transaction; never calls FCM)
        ▼
notification_outbox  (PENDING)                ◀— rolled back together with the business change
        │  NotificationOutboxWorker (own thread, polls every 5 s)
        ▼
NotificationOutboxProcessor.claimOutboxBatch   (FOR UPDATE SKIP LOCKED → PROCESSING)
        ▼
NotificationRecipientResolver  → recipients (tenant-scoped, actor removed)
        ▼
NotificationTemplateFactory → title/message per persona
        ▼
notification (history, 1 row / recipient)  +  notification_delivery (1 row / active device)   [one transaction per
outbox row]
        ▼
NotificationOutboxProcessor.claimDeliveries → FcmSender.send   (outside any DB transaction)
        ▼
Firebase Cloud Messaging (HTTP v1, Firebase Admin SDK)
        ▼
Android device — foreground: FlutterLocalNotifier (local notification)
              └ background / terminated: system tray notification (FCM `notification` block)
        ▼ tap
NotificationNavigationService (queued until session restored) → NotificationDestinationResolver (role-aware)
        ▼
existing GarageST screen (re-loads its data through the authenticated REST API)
```

Design decisions (see ADR-008): PostgreSQL is the source of truth for notification history; FCM is delivery only; a **transactional PostgreSQL outbox** gives durability; **Kafka/RabbitMQ were not introduced**; no database triggers; no "watch every UPDATE" logic — events are emitted only from meaningful business transitions. Delivery is **at-least-once**; the client de-duplicates by notification id.

28A.2 Backend components (`src/main/java/com/garageos/...`)

Concern	Class (path under `modules/notification/` unless noted)	Role
Enums	`core/enums/notification/{NotificationEventType, NotificationCategory, NotificationPriority, NotificationDeliveryStatus, NotificationOutboxStatus, DevicePlatform}`	closed vocabularies; `NotificationEventType` carries category, priority and deep-link entity type
Event	`event/NotificationEvent` (+ `NotificationFacts` keys)	immutable event: type, garageId, entityType/Id, jobCardId, discriminator, id-only facts; `getEventKey()`
Publisher	`event/NotificationEventPublisher`, helper `event/NotificationEvents`	inserts the outbox row in the **caller's** transaction (default `REQUIRED`, never `REQUIRES_NEW`); no-op when `notifications.enabled=false`; null-safe helper so services built without a publisher (unit tests) emit nothing
Recipients	`service/NotificationRecipientResolver` (+ `NotificationRecipient`, `NotificationPersona`)	the single place that decides who is notified (28A.5)
Text	`service/NotificationTemplateFactory`	persona-specific title/message; only the job-card number is interpolated
History	`service/NotificationService` / `impl/NotificationServiceImpl`	list (page ≤ 50), unread count, mark read, mark all read — always scoped to the caller
Devices	`service/DeviceTokenService` / `impl/DeviceTokenServiceImpl`	register (upsert + reassignment), unregister (ownership-checked)
Persistence	`entity/{DeviceToken, Notification, NotificationOutbox, NotificationDelivery}`, `repository/*`	native `INSERT … ON CONFLICT DO NOTHING … RETURNING id` and `SELECT … FOR UPDATE SKIP LOCKED` queries
Worker	`worker/NotificationOutboxWorker`, `worker/NotificationOutboxProcessor`	scheduling thread + the short transactional steps
FCM	`fcm/{FcmSender, FirebaseFcmSender, NoopFcmSender, FcmConfig}`	`FcmConfig` picks the sender; startup never fails for missing credentials
API	`controller/{NotificationController, DeviceTokenController}`, `dto/*`	see §27 *Notifications*
Config	`config/NotificationProperties`	all values from `application.properties` placeholders

28A.3 Flutter components (`lib/`)

Concern	File → class	Role
Payload	`core/push/push_payload.dart` → `PushPayload`	identifier-only data (`notificationId, type, entityType, entityId, jobCardId, garageId`); `decode/encode` for local-notification payloads
Firebase bootstrap	`core/push/firebase_push_gateway.dart` → `FirebaseBootstrap.initialize()`, `firebaseMessagingBackgroundHandler`	`Firebase.initializeApp()` in `main()`; failure (no `google-services.json`) leaves push disabled, app unaffected. Background handler is a deliberate no-op
Transport adapter	same file → `FirebasePushGateway` implements `PushGateway` (`core/push/push_gateway.dart`; `DisabledPushGateway` when Firebase is absent)	token, permission, refresh stream, foreground/opened streams, initial message, `deleteToken`
Local notifications	`core/push/local_notifier.dart` → `LocalNotifier`, `FlutterLocalNotifier`, `NotificationChannels`	creates channels, shows foreground notifications, reports taps, reads the launch payload
Orchestration	`core/push/push_notification_service.dart` → `PushNotificationService`	token lifecycle, foreground handling, tap routing; registered `lazy:false` in `app.dart`
Navigation	`core/push/notification_navigation_service.dart` → `NotificationNavigationService`	owns the global `navigatorKey` + route observer; queues taps; ownership check via mark-read
Routing table	`core/push/notification_destination_resolver.dart` → `NotificationDestinationResolver`	(type, entity, persona) → existing route
API	`features/notification/services/notification_service.dart` → `NotificationService`	`/notifications/**` calls through `ApiClient` (new `ApiClient.patch`)
State	`features/notification/state/notification_state.dart` → `NotificationState` (ChangeNotifier)	unread count + paged history; cleared on logout

Concern	File → class	Role
UI	`features/notification/screens/notifications_screen.dart`, `widgets/notification_bell_button.dart`, `screens/customer_job_card_redirect_screen.dart`	history list, bell + badge in the five home shells, customer job-card landing
Auth hook	`features/auth/state/auth_state.dart` → `AuthState.onBeforeLogout`	runs before the session is cleared (8 s cap; failures ignored)

28A.4 Event matrix (32 events — verified against `NotificationEventType`)

"Source" is where the transition is finalised and the publish call sits. *Discriminator* is the part of the idempotency key that separates repeated passes through the same state (28A.7). **Recipients** use the personas of 28A.5; the acting user is always removed. Deep links are per persona in 28A.11.

#	Event	Source (class.method)	Recipients	Category / priority	Entity	Discriminator
1	`BOOKING_REQUESTED`	`BookingServiceImpl.createBooking`	Owner, Manager/SA	BOOKING / NORMAL	BOOKING	`"1"`
2	`BOOKING_CREATED_BY_STAFF`	`BookingServiceImpl.createPhoneBooking`	Customer	BOOKING / LOW	BOOKING	`"1"`
3	`BOOKING_CONFIRMED`	`BookingServiceImpl.confirmBooking`	Customer	BOOKING / NORMAL	BOOKING	`"1"`
4	`BOOKING_REJECTED`	`BookingServiceImpl.rejectBooking`	Customer	BOOKING / NORMAL	BOOKING	`"1"`
5	`BOOKING_CANCELLED`	`BookingServiceImpl.cancelBooking`	Customer, Manager/SA	BOOKING / NORMAL	BOOKING	`"1"`
6	`PICKUP_DRIVER_ASSIGNED`	`NavigationTripServiceImpl.assignDriver`	Driver, Customer	TRIP / HIGH	TRIP	`"1"`
7	`TRIP_ACCEPTED`	`NavigationTripServiceImpl.acceptTrip`	Customer, Manager/SA	TRIP / NORMAL	TRIP	`"1"`
8	`TRIP_STARTED`	`NavigationTripServiceImpl.startTrip` and `continueTrip` (return leg)	Customer, Manager/SA	TRIP / NORMAL	TRIP	`"out"` / `"return"`
9	`DRIVER_ARRIVED`	`NavigationTripServiceImpl.arriveAtDestination` (not on a pickup's garage-side arrival)	Customer	TRIP / HIGH	TRIP	current leg
10	`HANDOVER_CODE_GENERATED`	`HandoverServiceImpl.getOrCreateActiveCode`	Driver	TRIP / NORMAL	TRIP	handover id
11	`HANDOVER_VERIFIED`	`HandoverServiceImpl.verify`	Customer, Manager/SA	TRIP / NORMAL	TRIP	handover id
12	`TRIP_COMPLETED`	`NavigationTripServiceImpl.completeTrip`	Customer, Manager/SA	TRIP / NORMAL	TRIP	`"1"`
13	`JOB_CARD_CREATED`	`JobCardServiceImpl.createJobCard`	Customer, Owner	JOB_CARD / NORMAL	JOB_CARD	`"1"`
14	`INSPECTION_STARTED`	`JobCardServiceImpl.startInspection`	Customer	INSPECTION / LOW	JOB_CARD	`"1"`
15	`INSPECTION_COMPLETED`	`JobCardServiceImpl.completeInspection`	Customer, Manager/SA	INSPECTION / NORMAL	JOB_CARD	`"1"`
16	`ESTIMATE_READY_FOR_APPROVAL`	`EstimateServiceImpl.updateEstimate` (job `ESTIMATE_PENDING` → `WAITING_FOR_APPROVAL`)	Customer	ESTIMATE / HIGH	ESTIMATE	transition time
17	`ESTIMATE_APPROVED`	`EstimateServiceImpl.approveEstimateCanonical`	Customer, Manager/SA, Owner	ESTIMATE / HIGH	ESTIMATE	transition time
18	`ESTIMATE_REJECTED`	`EstimateServiceImpl.rejectEstimate` (**now** `@Transactional`)	Manager/SA, Owner	ESTIMATE / HIGH	ESTIMATE	transition time

#	Event	Source (class.method)	Recipients	Category / priority	Entity	Discriminator
19	`REPAIR_TASK_ASSIGNED`	`JobAssignmentServiceImpl.assignJob` / `reassignJob` (TECHNICIAN type)	Technician	ASSIGNMENT / HIGH	REPAIR_TASK	assignment id
20	`REPAIR_TASK_STARTED`	`RepairTaskServiceImpl.startRepair`, `JobAssignmentServiceImpl.startJob`	Manager/SA	REPAIR / LOW	REPAIR_TASK	task `startedAt`
21	`REPAIR_TASK_COMPLETED`	`RepairTaskServiceImpl.completeRepair`, `JobAssignmentServiceImpl.completeJob`	Manager/SA	REPAIR / LOW	REPAIR_TASK	task `completedAt`
22	`JOB_REPAIR_STARTED`	`JobCardServiceImpl.startRepair` (manager confirmation)	Customer	REPAIR / NORMAL	JOB_CARD	transition time
23	`JOB_REPAIR_COMPLETED`	`RepairTaskServiceImpl.completeRepair`, `JobAssignmentServiceImpl.completeJob`, `JobCardServiceImpl.completeRepair` — **only when the all-tasks-complete transition actually happens**	Customer, Manager/SA, Owner	REPAIR / NORMAL	JOB_CARD	transition time
24	`QUALITY_CHECK_STARTED`	`ServiceWorkflowServiceImpl.performQualityCheck`	Manager/SA	QUALITY_CHECK / LOW	JOB_CARD	transition time
25	`QUALITY_CHECK_PASSED`	`QualityCheckServiceImpl.passQualityCheck`	Manager/SA	QUALITY_CHECK / NORMAL	JOB_CARD	transition time
26	`QUALITY_CHECK_FAILED`	`QualityCheckServiceImpl.failQualityCheck` (technicians captured **before** the task reset)	Technician, Manager/SA, Owner	QUALITY_CHECK / HIGH	JOB_CARD	transition time
27	`INVOICE_GENERATED`	`InvoiceServiceImpl.createInvoice`, `generateInvoice`	Customer, Manager/SA, Owner	INVOICE / HIGH	INVOICE	`"1"`
28	`INVOICE_ACCEPTED`	`InvoiceServiceImpl.acceptInvoice`	Manager/SA, Owner	INVOICE / NORMAL	INVOICE	`"1"`
29	`PAYMENT_RECEIVED`	`InvoiceServiceImpl.receivePayment`	Customer, Manager/SA, Owner	PAYMENT / NORMAL	INVOICE	`"1"`
30	`VEHICLE_READY_FOR_DELIVERY`	`JobCardServiceImpl.readyForDelivery` (also reached from `receivePayment`)	Customer, Manager/SA, Owner	DELIVERY / HIGH	JOB_CARD	transition time
31	`VEHICLE_DELIVERED`	`DeliveryServiceImpl.createDelivery`, `completeDeliveryFromTrip`	Customer, Manager/SA, Owner	DELIVERY / NORMAL	JOB_CARD	`"1"`
32	`JOB_CLOSED`	`JobCardServiceImpl.closeJobCard`	Owner	JOB_CARD / LOW	JOB_CARD	`"1"`

Manager/SA = users with role `MANAGER` or `SERVICE_ADVISOR` of the event's garage. Entity ids for ESTIMATE/INVOICE events are the estimate/invoice ids; the job-card id travels separately. Event identity includes the entity id (e.g. `ESTIMATE_APPROVED:ESTIMATE:7:<discriminator>`).

Not events (not represented in code): `PAYMENT_FAILED`, refund/partial payment (those `PaymentStatus` values are never written), job-card cancellation (`CANCELLED` is never set by a service), estimate expiry. `generateInvoiceByEstimateId` (no controller calls it; creates DRAFT invoices) emits nothing.

28A.5 Recipients and personas

Persona	Resolved from	Notes
Customer	the event's own `customerId` → `customer.mobile_number` → `users.mobile`; user must be `ACTIVE` with role `CUSTOMER`	there is **no foreign key** between customer and user. A customer without an application account receives **no push** (and no in-app history); no error, no SMS/WhatsApp fallback. A staff user who shares the mobile number is not treated as the customer (role check)
Owner	`garage.owner_user_id` of the event's garage (must be `ACTIVE`)	receives the business-level events only (see matrix)
Manager and **Service Advisor**	`UserRepository.findByGarageIdAndRoleAndStatus(garageId, role, ACTIVE)` for `MANAGER` and `SERVICE_ADVISOR`, re-filtered on `user.garageId == garageId`	the same tenant source the security principal uses (`users.garage_id`)
Technician	user ids stored in the event facts at emit time (`technicianUserIds`): the assignee for `REPAIR_TASK_ASSIGNED`; for `QUALITY_CHECK_FAILED` the distinct non-cancelled assignees captured **before** `failQualityCheck` clears task assignments	must belong to the event's garage
Driver	`NavigationTrip.driverId` (fact `driverUserId`)	must belong to the event's garage

Roles that intentionally receive **no** automatic events: `ACCOUNTANT`, `CASHIER`, `INVENTORY_MANAGER` (and `SUPER_ADMIN`/`USER`). **Actor self-notification is suppressed:** the publisher records `actorUserId` from the security context; the resolver removes that user (e.g. a manager approving an estimate is not told about it; the other managers and the owner are). Recipients are de-duplicated; the first matching persona wins (customer, technician, driver, owner, manager/SA).

28A.6 Multi-garage isolation

Every staff recipient (owner, manager, service advisor, technician, driver) must satisfy `user.garage_id == event.garageId`. This is enforced in `NotificationRecipientResolver` even for user ids supplied through event facts: a technician or driver id belonging to another garage is **rejected** (`NotificationRecipientResolverTest`: `technician_isResolvedFromFacts_onlyWithinGarage`, `driver_isResolved_withinGarageOnly`, `crossGarage_staffOfGarageB_neverReceiveGarageAEvents`). The resolver never reuses the unscoped `getTechnicians()` / `getRepairEmployees()` queries. Customers are cross-garage records and receive events only for their **own** customer id (they may legitimately receive notifications from several garages). FCM topics are not used; recipient selection happens only on the server.

28A.7 Idempotency

- **Event key** = `TYPE:ENTITY_TYPE:ENTITY_ID:discriminator`. `notification_outbox.event_key` is `UNIQUE`, so republishing the same logical event is a no-op (`INSERT … ON CONFLICT DO NOTHING`).
- **Per recipient:** `notification (event_key, user_id)` is `UNIQUE` — a worker replay cannot create a second history row. `notification_delivery (notification_id, device_token_id)` is `UNIQUE`.
- **Why a discriminator:** a job card can legitimately pass through the same state more than once (a failed quality check returns it to `REPAIR_PENDING`, so `JOB_REPAIR_COMPLETED` happens again). A key of only (type, entity id) would wrongly suppress the second, real event. Re-enterable transitions therefore use the transition timestamp; one-off events use `"1"` (assignment/handover events use their own row id).
- **Guarantee:** at-least-once. A crash after FCM accepts a message but before the delivery row is updated can re-send once; the app suppresses duplicates by notification id.
- Test evidence: `NotificationPipelineIT.businessCommit_persistsOutboxRow_andReplayIsDeduplicated`, `fullPipeline_createsOneNotificationAndDelivery_andSecondRunSendsNothingMore`; `NotificationEventPublisherTest`.

28A.8 Outbox, worker and retry (values from `NotificationProperties`)

- **Outbox states:** `PENDING → PROCESSING → SENT` (processed; recipients and delivery rows created) or `FAILED` (attempts exhausted). **Delivery states:** `PENDING → SENT | FAILED | INVALID_TOKEN`.
- **Atomicity:** the outbox row commits with the business change or rolls back with it (`NotificationPipelineIT.businessRollback_leavesNoOutboxRow`). FCM is **never** called inside a business transaction; an FCM outage cannot roll back a business operation.

- **Worker:** `NotificationOutboxWorker` runs on its **own single-thread executor** (not Spring's shared scheduler, so it cannot delay `MediaUploadRetryScheduler`); started only when `notifications.enabled=true`. Poll every **5 000 ms** (`notifications.worker.delay-ms`), batch **50** (`batch-size`).
- **Claiming:** `SELECT … FOR UPDATE SKIP LOCKED` on due rows, marked `PROCESSING` with `locked_at`. **Stale recovery:** `PROCESSING` rows older than **5 minutes** (`stale-lock-minutes`) return to `PENDING`.
- **Fan-out:** one transaction per outbox row resolves recipients, inserts `notification` rows and one `notification_delivery` per active device token (only when push is available). A failure rolls the whole row back and schedules a retry.
- **Retry/backoff:** `base-backoff-seconds` = **30**, doubling per attempt (30 s, 60 s, 2 min, 4 min, 8 min…), capped at **3 600 s**; `max-attempts` = **6**, then `FAILED`. Applies to outbox rows and to deliveries.
- **Delivery lease:** a claimed delivery is invisible for **2 minutes** so a crashed worker's delivery becomes due again.
- **FCM errors:** `UNREGISTERED` / `SENDER_ID_MISMATCH` → delivery `INVALID_TOKEN` and the device token is deactivated (never retried); `UNAVAILABLE` / `INTERNAL` /quota/network → retried with backoff; authentication/permission errors → treated as configuration errors: retried (bounded) and the worker stops sending for that tick instead of hammering FCM. One bad token never blocks the user's other devices.
- **Disabled / no credentials:** `notifications.enabled=false` → publisher is a no-op, worker never starts, `NoopFcmSender`. Enabled but no `FIREBASE_CREDENTIALS_BASE64` (or invalid) → `NoopFcmSender`: in-app history is still produced, **no deliveries are created**, the application still starts.

28A.9 Firebase / FCM

- **Firebase Cloud Messaging**, **Firebase Admin SDK** (`com.google.firebase:firebase-admin` **9.9.0**), HTTP v1 API, Android delivery. Connect timeout 10 s, read timeout 15 s.
- **Server configuration — variable names only:** `NOTIFICATIONS_ENABLED` (default `false`) → `notifications.enabled`; `FIREBASE_CREDENTIALS_BASE64` → `notifications.fcm.credentials-base64` (Base64 of the Firebase service-account JSON). Never commit or document the values.
- **Client configuration:** `android/app/google-services.json` (git-ignored, one per environment) + Google Services Gradle plugin (§28A.12). Without it the app builds and runs with push inactive.
- **Message shape:** an FCM `notification` block (title, body — so the tray shows it when the app is backgrounded/killed) plus a `data` map of identifiers only: `notificationId`, `type`, `entityType`, `entityId`, `jobCardId`, `garageId`. No customer names/phones, amounts, tokens or credentials.
- **Priority → channel:** HIGH → Android priority `HIGH`, channel `garagest_urgent`; NORMAL → `garagest_default`; LOW → `garagest_low`. HIGH is reserved for events that need action or are time-sensitive (see the matrix).
- **Credential failure behaviour:** see 28A.8 — never affects business transactions; logged without secrets.

28A.10 Flutter push lifecycle

App state	What happens
Foreground	`FirebaseMessaging.onMessage` → `PushNotificationService` de-duplicates by notification id → `FlutterLocalNotifier.show` (stable local id = notification id, so repeats replace) on the matching channel → `NotificationState.onPushReceived()` refreshes the badge (and the list if loaded)
Background	the system shows the FCM `notification` block in the tray; the Dart background handler does nothing (no UI, providers or network). Tap → `onMessageOpenedApp` → `NotificationNavigationService.handleTap`
Terminated / cold start	`getInitialMessage()` (FCM) or the local launch payload is captured at start and **queued**. It is dispatched only after (1) the session is restored, (2) a user is signed in, (3) the app has left splash/login (`routeChanged` observer). Splash's role redirect is therefore never overridden

Tap handling: (1) `markRead(notificationId)` through the API — this doubles as the **ownership check**: HTTP 404 means the notification is not the signed-in user's (e.g. a tap that arrives after an account switch) and **nothing opens**; a network error still opens the screen because the destination re-checks access; (2) the persona's destination is resolved; (3) navigation is deferred one event-loop turn (never from inside a navigator callback). **Duplicate taps** of the same notification within 5 s are ignored; a queued tap older than 30 min expires. A tap while logged out is held until login.

History and badge: `NotificationsScreen` (`/notifications`) shows title, message, relative time and an unread dot; paged (20 per page), pull-to-refresh, mark read on tap, mark all read. `NotificationBellButton` (badge, "99+" cap) is in the AppBar of the Employee/Manager, Technician, Driver, Owner and Customer home shells and reads the shared `NotificationState` (no per-widget polling). `NotificationState.clear()` runs on logout.

Permission: the Android 13+ runtime permission is requested once per app run after login; denial is respected and the app continues to work (history still loads).

28A.11 Notification tap → destination (`NotificationDestinationResolver`)

Persona	Entity	Destination (existing route)
Customer	BOOKING	`Routes.bookingDetail` (booking id)
Customer	TRIP	`DRIVER_ARRIVED` → `Routes.handoverCode`; `HANDOVER_VERIFIED` → `Routes.handoverStatus`; otherwise `Routes.activeTrip` (trip id)
Customer	ESTIMATE / INVOICE	new `Routes.customerEstimateDetail` / `Routes.customerInvoiceDetail` (entity id)
Customer	JOB_CARD / REPAIR_TASK	new `Routes.customerJobCard` → `CustomerJobCardRedirectScreen` (resolves the job-card **number** from the customer's own job cards, then opens `CustomerRepairTrackingScreen`)
Owner, Manager/SA	BOOKING	`Routes.garageBookingQueue`
Owner, Manager/SA	TRIP	`Routes.managerFleetMap`
Owner, Manager/SA	JOB_CARD / ESTIMATE / INVOICE / REPAIR_TASK	`Routes.jobCardDetail` (job-card id)
Technician	any	`Routes.technicianHome` (there is no repair-task screen route)
Driver	TRIP	`HANDOVER_CODE_GENERATED` → `Routes.handoverVerify`; `PICKUP_DRIVER_ASSIGNED` → `Routes.driverHome`; otherwise `Routes.activeTrip`
any	missing ids	`Routes.notifications` (history)
unknown persona	—	nothing

The payload is routing information, **not authorization**: every destination loads its data through the normal authenticated API, whose checks are unchanged.

28A.12 Device-token lifecycle

A device token identifies an **app installation**, not a user; it is attached to whichever authenticated user most recently registered it.

Step	Behaviour
Login / session restore	`PushNotificationService` observes `AuthState`; when signed in it requests permission (once per run), reads the FCM token and calls `POST /notifications/devices`
Registration	backend upsert by token: new row, or update + reactivate; **if the token belongs to another user it is reassigned to the caller** (account-switch safety net). The user comes only from the JWT
FCM token refresh	`onTokenRefresh` → register again (only while signed in)
Logout (required order)	`AuthState.logout()` → hook `PushNotificationService.prepareForLogout`: (1) `POST /notifications/devices/unregister` while the access token is still valid (token in the **body**, never the URL), (2) `FirebaseMessaging.deleteToken()`, (3) existing `clearSession()`. Each step is best-effort with a timeout; **logout always completes**. `NotificationState` is cleared
Account switching	the next login registers the (new) token for the new user; a stale token still tied to the old user is reassigned by the upsert
Multiple devices	one `device_token` row per installation; fan-out creates one delivery per active token; per-device status
Invalid token	`UNREGISTERED` → delivery `INVALID_TOKEN`, token `is_active=false`
Session expiry	the client has no refresh interceptor (known), so a token cannot be unregistered when a session simply expires; reassignment on next register and invalid-token pruning cover this. **No time-based pruning by `last_seen_at` exists**

28A.13 Android

- **Gradle:** `android/settings.gradle.kts` declares `com.google.gms.google-services` (4.4.4, `apply false`); `android/app/build.gradle.kts` applies it **only if** `android/app/google-services.json` **exists**, enables core-library desugaring (`isCoreLibraryDesugaringEnabled`, `desugar_jdk_libs:2.1.4` — required by `flutter_local_notifications`).

- **Dependencies (`pubspec.yaml`):** `firebase_core ^4.15.0`, `firebase_messaging ^16.7.0`, `flutter_local_notifications ^22.3.1`.
- **Manifest:** `POST_NOTIFICATIONS` (declared earlier for the location foreground service), FCM `default_notification_channel_id = garagest_default`, `default_notification_icon = @mipmap/ic_launcher`.
- **Channels (created at startup by `FlutterLocalNotifier`):** `garagest_urgent` (high importance), `garagest_default`, `garagest_low`. The geolocator foreground-service notification is unaffected. The OS and the user ultimately control permission, sound and vibration per channel.
- **Tested platform:** Android (physical device). **iOS: NOT IMPLEMENTED** — no APNs setup, entitlements, Podfile or Firebase iOS config; the push layer is behind `PushGateway` so it could be added.

28A.14 Security

- Identity for every notification endpoint is the **JWT principal**; `user_id` is never accepted from the client. All history queries are `WHERE user_id = caller`; mark-read looks up `(id, user_id)` so another user's id behaves as not found (`NotificationServiceImplTest`).
- Device unregister only deactivates a token owned by the caller; registration reassigns by token without trusting a client-supplied owner.
- Staff recipients are garage-scoped (28A.6); customers only for their own entities.
- FCM payload = identifiers only; message text carries only the job-card number.
- Service-account credentials only through `FIREBASE_CREDENTIALS_BASE64`; never in source, logs or Flutter. **Raw FCM tokens are never logged** (logs carry ids: `deviceTokenId`, `notificationId`, `deliveryId`).
- Log tags: `[NOTIF_OUTBOX]`, `[NOTIF_RESOLVE]`, `[NOTIF_FCM]`, `[NOTIF_DEVICE]`.

28A.15 Validation

Automated (this repository, 2026-10-02 run — see §37): backend unit tests for the publisher (7), recipient resolver (13), template factory (4), device tokens (7), history service (5), outbox processor, worker, estimate wiring; real-database `NotificationPipelineIT` (6); Flutter tests for payload parsing, role-aware routing, queued navigation, token lifecycle, logout ordering (real `AuthState`), foreground handling and the API contract. **These use a stub FCM sender / fake gateway.**

PHYSICALLY VALIDATED — ANDROID (product owner, physical device, real Firebase/FCM):

- the notification appears and plays its sound; vibrate/silent behaviour follows the device mode and settings
- it appears in the Android system notification tray
- the notification history is visible inside GarageST, and the unread badge/history behave correctly
- tapping a notification opens GarageST and **deep-link navigation works**
- *Vehicle Ready for Delivery* and *Payment Received* notifications were observed, and the notification content includes the **job-card context**

Not claimed: every one of the 32 events has **not** been individually exercised on a device; iOS has not been tested (not implemented); multi-device, force-kill/cold-start per persona, permission-denied and account-switch scenarios are **pending manual confirmation** (§38 rows `NT-*`).

28A.16 Known limitations

Limitation	Impact	Workaround	Future improvement
iOS push NOT IMPLEMENTED	iPhone users receive nothing	in-app history still works where the app runs	add APNs + Firebase iOS config behind `PushGateway`
Customer ↔ user match is **by mobile number** (no FK)	a customer without an account, or whose mobile differs from the account's, gets nothing	keep mobile numbers consistent	explicit customer–user link
Backend `AuthServiceImpl.logout` looks the session up by the access token and so never revokes the session (**legacy issue, not changed**)	server-side session stays valid after logout; device-token cleanup is therefore an explicit separate call	none needed for notifications	fix as a separately approved auth task
No refresh-token interceptor in Flutter	a token cannot be unregistered when a session just expires	reassignment on next register; invalid-token pruning	stale-token cleanup by `last_seen_at`; refresh interceptor (approval-gated)

Limitation	Impact	Workaround	Future improvement
Notification small icon reuses `@mipmap/ic_launcher`	may render as a white square on some Android versions	none	dedicated monochrome icon
Automated tests use a stub FCM sender	real FCM behaviour is covered by the physical Android test only	physical test matrix §38 (`NT-*`)	optional staging project/contract test
`InvoiceServiceImpl.createInvoice` has no `@Transactional` (pre-existing)	its outbox row is not atomic with the invoice save on that path (the workflow path `generateInvoice` is transactional)	use the workflow path	add the annotation (approval-gated)
`appVersion` is not sent by the app	`device_token.app_version` stays empty	—	add a package-info dependency
No retention/cleanup job for outbox, deliveries or notifications	tables grow	manual SQL	scheduled cleanup (e.g. outbox/delivery 30–90 days)
No per-user preferences / quiet hours	every relevant event is pushed	OS channel settings	preferences
Worker polls every 5 s	up to ~5 s latency before a push is sent	lower `delay-ms`	`LISTEN/NOTIFY` or immediate nudge
Single backend instance assumed for the worker executor	multi-instance is safe for claiming (`SKIP LOCKED`) but untested	—	load test
No `PAYMENT_FAILED`/refund/cancellation events	not applicable today (no gateway, no cancel flow)	—	add with those features

29. Flutter Screen / Route Map

Screen	Route	Entered by / how	State consumed	API calls	Changes	Next
`EmployeeHomeScreen`	`/employee/home`	splash redirect (employee roles)	`Permissions`, dashboards	dashboard + job list services	`WorkflowState.reset()` on FAB	workflow shell
`WorkflowShellScreen`	`/employee/workflow` (arg: job number | `WorkflowStartAtJobStep` | none)	FAB, Job Card actions, Booking "Create Job Card"	`WorkflowController`, `WorkflowState`	`GET /workflow/{n}/resume` when resuming	step position	its step screens
`SearchScreen`	(shell step)	first step	`WorkflowState.customer/vehicle`	customer by mobile, vehicle by registration	sets customer/vehicle	Job Card step
`JobStepScreen`	(shell)	manager/SA	customer, vehicle, `bookingId`	`POST /workflow/job`, `/inspection/start`, media queue	creates job, uploads media (queue)	Inspection
`InspectionScreen`	(shell)	manager	complaints, job	inspection create	inspections	Estimate
`EstimateScreen`	(shell)	manager	inspections	`/inspection/complete`, `/estimate`, `POST /estimates`	estimate DRAFT	Estimate Items
`EstimateItemScreen`	(shell)	manager	estimate	estimate-item CRUD	items, totals	Summary
`EstimateSummaryScreen`	(shell)	manager	estimate, items	`PUT /estimates/{id}`	job `WAITING_FOR_APPROVAL`	Approval (via `initializeFromWorkflowStatus`)

Screen	Route	Entered by / how	State consumed	API calls	Changes	Next
`ApprovalScreen`	(shell)	manager	`estimate.status`, `workflowStatus.status`	pull-to-refresh: `GET /workflow/{n}/resume`; button: `POST /workflow/{n}/repair/start`	`REPAIR_IN_PROGRESS`	Repair (re-derived)
`RepairScreen`	(shell)	manager, technician	tasks, assignments	`/job-assignments/*`, `/repair-tasks`, `/workflow/{n}/quality-check`	assignments/tasks; job `REPAIR_COMPLETED` (auto) then QC	Quality Check
`QualityCheckScreen`	(shell)	manager/technician	job	QC pass/fail	`READY_FOR_INVOICE` / `REPAIR_PENDING`	Invoice
`InvoiceScreen`	(shell)	manager/accountant	estimate, invoice	`/workflow/{n}/invoice`, `/invoice/accept`	`INVOICE_GENERATED`	Payment
`PaymentScreen`	(shell)	manager/accountant/cashier	invoice	`/workflow/{n}/payment`	`READY_FOR_DELIVERY`	Delivery
`DeliveryScreen`	(shell)	manager	`workflowStatus`, `jobCardId`, invoice	`GET /navigation/requests/job-card/{id}/delivery`, `POST /deliveries` (collection), `PUT /jobcards/{n}/close` via `closeJob`, `resumeWorkflow`	request/assignment, delivered, closed	`Routes.activeTrip`, sheets
`JobCardDetailScreen`	`/job-card` (arg: id)	job lists	`JobCardView`	projection, `JobCardActionService`, media	transitions per action	workflow shell
`BookingDetailScreen`	`/booking/detail` (arg: id)	queue, my bookings, journey	booking + pickup trip	bookings, `getTripByRequest`, `CustomerService`, `VehicleRepository`	confirm/reject/cancel; loads customer/vehicle into `WorkflowState`	`AssignDriverSheet`, `Routes.activeTrip`, workflow shell (`WorkflowStartAtJobStep`)
`DriverDashboard` (in `DriverHomeScreen`)	`/driver/home`	splash (driver role)	trips (`nextDriverAction`)	`getMyTrips`, accept/start/arrive/**complete**	trip state	`TripEvidenceCaptureScreen`, `Routes.handoverVerify`, `Routes.activeTrip`
`ActiveTripScreen`	`/navigation/active-trip` (arg: tripId)	driver, customer, manager, booking, delivery step	`LiveTripState` (trip, location, route), garage	trip GET/poll, location, routes, STOMP, `/garages/{id}`	publishes driver GPS (driver)	`Routes.handoverCode`
`TripEvidenceCaptureScreen`	pushed directly (tripId, stage)	driver "Capture Evidence"	captured shot types	trip media GET/POST	trip media	back

Screen	Route	Entered by / how	State consumed	API calls	Changes	Next
`HandoverCodeScreen`	`/handover/code` (arg: tripId)	customer/ `ActiveTripScreen`	trip	`GET …/handover/code`	pending handover	back
`HandoverVerifyScreen`	`/handover/verify` (arg: tripId)	driver "Confirm Handover"	trip	`POST …/handover/verify`, then `continue` (PICKUP outbound) or `complete` (DELIVERY)	handover `VERIFIED`, trip	back
`LocationPickerScreen`	pushed directly	`PickupLocationSelector`, `CreateDeliveryRequestSheet`	initial `PickupLocation`	place search, reverse geocode	none	pops `PickupLocation`
`CreateDeliveryRequestSheet`	bottom sheet	Delivery step	customer/vehicle/garage/job ids	`POST /navigation/requests`	delivery request	returns request id
`AssignDriverSheet`	bottom sheet	booking detail, delivery step	request id	`GET /users/drivers`, `POST /navigation/trips`	trip created	returns bool
`CustomerEstimateDetailScreen`	customer portal	customer estimate list	estimate	`GET` estimate, `PUT /estimates/{id}/approve|reject`	estimate `APPROVED`, job `REPAIR_PENDING`	refresh
`LiveVehicleJourneyScreen`	`/customer/vehicle-journey`	customer home	journey summaries	`GET /customer/vehicle-journeys/active`	none	`BookingDetailScreen`, `Routes.activeTrip`
`ManagerFleetMapScreen`	`/navigation/fleet-map`	manager	fleet	fleet endpoint	none	`Routes.activeTrip`
`NotificationsScreen`	`/notifications`	bell button in every home shell; notification tap fallback	`NotificationState`	`GET /notifications`, `/unread-count`, `PATCH …/read`, `/read-all`	read flags, badge	destination of the tapped notification (§28A.11)
`NotificationBellButton`	(widget)	AppBar of Employee/Manager, Technician, Driver, Owner, Customer home shells	`NotificationState.unreadCount`	none (shared state)	none	`Routes.notifications`
`CustomerEstimateDetailScreen` / `CustomerInvoiceDetailScreen` (new routes)	`/customer/estimate`, `/customer/invoice` (arg: id)	notification tap (customer)	portal data	existing portal calls	as before	—
`CustomerJobCardRedirectScreen`	`/customer/job-card` (arg: jobCardId)	notification tap (customer)	—	`GET /customer/jobcards`	none	`CustomerRepairTrackingScreen`

30. Key Code Paths

Notation: *Flutter screen → service → endpoint → controller → service → entity/state*.

1. Customer approval → manager confirmation. BEFORE: `resolveCompleted` listed `APPROVAL` at `REPAIR_PENDING`, so the manager was moved to Repair. AFTER: `APPROVAL` only from `REPAIR_IN_PROGRESS`. Path: `CustomerEstimateDetailScreen` → `CustomerPortalService.approveEstimate` → `PUT /estimates/{id}/approve` → `EstimateController` → `EstimateServiceImpl.approveEstimate(Long)` → `approveEstimateCanonical` → `Estimate.APPROVED`, `createRepairTasks`, `JobCard.REPAIR_PENDING`. Manager: `ApprovalScreen._refresh` → `WorkflowService.resumeWorkflow` → `GET /workflow/{n}/resume` → `ServiceWorkflowServiceImpl.resumeWorkflow` → `WorkflowController.initializeFromWorkflowStatus` stays on Approval; `_proceedToRepair` → `WorkflowService.proceedToRepair` → `POST /workflow/{n}/repair/start` → `ServiceWorkflowServiceImpl.startRepair` → `JobCardServiceImpl.startRepair` → `JobCard.REPAIR_IN_PROGRESS` → refresh → step = Repair.

2. Repair-start bypass prevention. BEFORE: `JobAssignmentServiceImpl.startJob` set `REPAIR_PENDING → REPAIR_IN_PROGRESS` itself (no manager/estimate/validator). AFTER: it throws *"A manager must proceed…"* unless the job is `REPAIR_IN_PROGRESS`, and never writes the job. Path: `RepairScreen._startRepair` → `JobAssignmentService.start` → `PUT /job-assignments/{id}/start` → `JobAssignmentController` → `JobAssignmentServiceImpl.startJob` → task/assignment `IN_PROGRESS`. `RepairTaskServiceImpl.startRepair` had the same guard already.

3. RepairTask completion. BEFORE: precondition in `RepairTaskServiceImpl.completeRepair` was commented out. AFTER: job must be `REPAIR_IN_PROGRESS`. Path: `RepairScreen._completeRepair` → `PUT /job-assignments/{id}/complete` (or `/repair-tasks/{id}/complete`) → task `COMPLETED`; last task → `statusValidator.validate` + job `REPAIR_COMPLETED`.

4. Booking → JobCard. BEFORE: Flutter never sent `bookingId`. AFTER: `BookingDetailScreen._startJobCard` → `WorkflowState.bookingId` → `JobStepScreen` → `CreateJobCardRequest.bookingId` → `POST /workflow/job` → `JobCardServiceImpl.createJobCard`/`resolveBooking` → `JobCard.bookingId`, Booking `COMPLETED`, `NavigationRequest.jobCardId` (pickup).

5. Pickup trip. `BookingDetailScreen` → `AssignDriverSheet` → `NavigationTripService.assignDriver` → `POST /navigation/trips` → `NavigationTripController` → `NavigationTripServiceImpl.assignDriver` → `NavigationTrip(ASSIGNED)`, request `ASSIGNED`. Driver: `DriverDashboard` → `acceptTrip`/`startTrip`/`arriveAtDestination` → `/accept /start /arrive`.

6. Pickup return completion. BEFORE: no UI called `completeTrip` for pickup (the handover screen re-called `continue`). AFTER: `DriverDashboard._DriverAction` (return leg + arrived) → `_complete` → `NavigationTripService.completeTrip` → `POST /trips/{id}/complete` → `completeTrip` → trip + request `COMPLETED`.

7. Pickup OTP. `ActiveTripScreen` → `HandoverCodeScreen` → `GET …/handover/code` → `HandoverServiceImpl.getOrCreateActiveCode` → `VehicleHandover(PENDING, codeHash)`; driver `HandoverVerifyScreen` → `POST …/handover/verify` → `verify` → `VERIFIED`; `_advance` → `continue`.

8. Trip ↔ JobCard linkage. `navigation_request.job_card_id` set by `JobCardServiceImpl.createJobCard` (pickup) and `NavigationRequestServiceImpl.createRequest` (delivery); exposed as `NavigationRequestResponse.jobCardId` and `NavigationTripResponse.jobCardId` (`NavigationTrip.jobCardId` in Flutter).

9. Delivery request. `DeliveryScreen._assignDeliveryDriver` → `CreateDeliveryRequestSheet.show` → `NavigationTripService.createDeliveryRequest` → `POST /navigation/requests` → `NavigationRequestController` → `NavigationRequestServiceImpl.createRequest` → `validateDeliveryRequest` → `NavigationRequest(DELIVERY, REQUESTED, jobCardId, address, lat, lng)`.

10. Delivery map selection. `CreateDeliveryRequestSheet._selectOnMap` → `LocationPickerScreen(mode: delivery)` → `PickupLocation` → fields → `resolveDeliveryAddress`.

11. Delivery driver assignment. `DeliveryScreen` → `AssignDriverSheet` → `POST /navigation/trips` (same as pickup; request must be `REQUESTED`). The screen re-loads the request afterwards.

12. Delivery navigation. Driver `ActiveTripScreen` → `DriverLocationTracker` → STOMP → `LiveTripState` on customer/manager; destination = `trip.destinationLatitude/Longitude` (delivery coordinates).

13. Delivery OTP. As #7 with `direction = DELIVERY`; `HandoverVerifyScreen._advance` → `completeTrip`.

14. Delivery evidence. `DriverDashboard` → `TripEvidenceCaptureScreen(stage: 'DELIVERY')` → `TripMediaService.upload` → `POST /trips/{id}/media` → `NavigationTripMediaServiceImpl.upload` (trip `IN_PROGRESS`, arrived) → `completeTrip` → `validateRequiredMedia(DELIVERY)`.

15. Delivery trip completion. `HandoverVerifyScreen._advance` → `POST /trips/{id}/complete` → `NavigationTripServiceImpl.completeTrip` → `setRequestStatus(COMPLETED)`.

16. Delivery → JobCard `DELIVERED`**.** `completeTrip` → `deliveryService.completeDeliveryFromTrip(jobCardId, driverName, "Customer (OTP verified)")` → `Delivery(DELIVERED)`, `JobCard.DELIVERED`.

17. Customer journey. `LiveVehicleJourneyScreen` → `GET /customer/vehicle-journeys/active` → `CustomerVehicleJourneyServiceImpl.getActiveJourneys` → linked-pickup rule (§22).

18. Workflow resume/state restoration. Job Card "Resume Workflow" / dashboard → `Routes.employeeWorkflow(jobNumber)` → `WorkflowShellScreen._resume` → `resumeWorkflow` → `syncWorkflowState` → `initializeFromWorkflowStatus`. The Delivery step additionally restores its request from the backend.

31. Sequence Diagrams

Diagram 1 — Normal job workflow

```
sequenceDiagram
  actor C as Customer
  actor M as Manager
  actor T as Technician
  participant F as Flutter
  participant B as Backend
  C->>B: Booking (POST /bookings)
  M->>B: confirm (POST /bookings/{id}/confirm)
  M->>F: Create Job Card (bookingId)
  F->>B: POST /workflow/job (bookingId)  → JobCard OPEN, Booking COMPLETED
  F->>B: POST /workflow/{n}/inspection/start → INSPECTION_PENDING
  F->>B: POST /workflow/{n}/inspection/complete + /estimate → ESTIMATE_PENDING
  M->>B: PUT /estimates/{id} → WAITING_FOR_APPROVAL
  C->>B: PUT /estimates/{id}/approve → Estimate APPROVED, tasks PENDING, REPAIR_PENDING
  M->>B: POST /workflow/{n}/repair/start → REPAIR_IN_PROGRESS  (manager gate)
  T->>B: job-assignments accept/start/complete → tasks COMPLETED → REPAIR_COMPLETED (auto)
  M->>B: POST /workflow/{n}/quality-check, QC pass → READY_FOR_INVOICE
  M->>B: POST /workflow/{n}/invoice → INVOICE_GENERATED
  M->>B: POST /workflow/{n}/payment → READY_FOR_DELIVERY
  Note over M,B: Delivery (Diagram 3) → DELIVERED
  M->>B: PUT /jobcards/{n}/close → CLOSED
```

Diagram 2 — Pickup

```
sequenceDiagram
  actor C as Customer
  actor D as Driver
  actor M as Manager
  participant B as Backend
  C->>B: Booking (pickupRequested)
  M->>B: confirm → NavigationRequest(PICKUP, REQUESTED)
  M->>B: POST /navigation/trips → Trip ASSIGNED, request ASSIGNED
  D->>B: accept, start → Trip IN_PROGRESS (GARAGE_TO_CUSTOMER), request IN_PROGRESS
  D->>B: arrive (arrivedAt)
  D->>B: 5 BEFORE_PICKUP photos
  C->>B: GET handover/code (OTP)
  D->>B: POST handover/verify {code} → VERIFIED
  D->>B: POST continue → CUSTOMER_TO_GARAGE (arrivedAt reset)
  D->>B: arrive at garage
  D->>B: POST complete → Trip COMPLETED, request COMPLETED
  M->>B: POST /workflow/job (bookingId) → JobCard; request.jobCardId set; Booking COMPLETED
```

Diagram 3 — Delivery

```
sequenceDiagram
  actor M as Manager
  actor D as Driver
  actor C as Customer
  participant F as Flutter
  participant B as Backend
  Note over B: JobCard READY_FOR_DELIVERY
  F->>B: GET /navigation/requests/job-card/{id}/delivery (404 = none)
  M->>F: Select location on map (LocationPickerScreen)
  F->>B: POST /navigation/requests (DELIVERY, jobCardId, address, lat, lng)
  M->>B: POST /navigation/trips (driver) → Trip ASSIGNED
  D->>B: accept, start; GPS → STOMP
  D->>B: arrive; 5 DELIVERY photos
  C->>B: GET handover/code
  D->>B: POST handover/verify → VERIFIED
  D->>B: POST /navigation/trips/{id}/complete
  B->>B: request COMPLETED; DeliveryService.completeDeliveryFromTrip
  B->>B: Delivery(DELIVERED); JobCard READY_FOR_DELIVERY → DELIVERED (one transaction)
  M->>B: PUT /jobcards/{n}/close → CLOSED
```

Diagram 4 — Media

```
sequenceDiagram
  participant F as Flutter queue
  participant B as Backend
  participant R as R2
  F->>B: POST upload-intent (stage, metadata)
  B-->>F: presigned PUT (+ thumbnail PUT for video), storageKey
  F->>R: PUT source (image) / evidence.mp4 + thumbnail.jpg (video)
  F->>B: POST /media/{id}/complete
  B->>R: (image) download source
  B->>R: upload _evidence.jpg + _thumbnail.jpg, delete _source
  B-->>B: status COMPLETED, storage_key = evidence
  F->>B: GET /media/{id}/access?variant=evidence|thumbnail
  B-->>F: short-lived presigned GET
  F->>R: GET (cached as media-evidence-{id} / media-video-{id})
```

Diagram 5 — Live navigation

```
sequenceDiagram
  participant G as Geolocator (10 m filter)
  participant D as DriverLocationTracker
  participant S as STOMP /ws
  participant B as Backend
  participant V as Customer/Manager LiveTripState
  G->>D: position
  D->>S: SEND /app/location/update
  S->>B: LocationWebSocketController → DriverLocationServiceImpl
  B-->>V: /topic/trips/{id}/location
  loop every 10 s (fallback)
    V->>B: GET /navigation/trips/{id} and /location
  end
  V->>B: POST /navigation/routes (only after ≥150 m movement)
```

32. Error Handling

Situation	Backend condition / message	Expected UI	Recovery
Manager confirmation rejected	non-manager: "Only a Manager can proceed a Job Card to repair."	snackbar with message	sign in as manager
Estimate not approved	"Estimate must be customer-approved before repair can start."	Approval button only shown when approved	wait for customer

Situation	Backend condition / message	Expected UI	Recovery
JobCard in wrong state	`IllegalStateException` "Invalid Job Card status transition: A -> B"	message; screen reloads (`JobCardActionsSection` refreshes on rejection)	refresh; state is stale
Technician starts too early	"A manager must proceed this Job Card to repair before a Repair Task can be started."	snackbar	manager confirms first
Complete task in wrong state	"Job Card must be in REPAIR_IN_PROGRESS before a Repair Task can be completed."	snackbar	check job status
Delivery request invalid	"A delivery request must reference the Job Card…", "…requires a delivery address with coordinates.", "Delivery can only be requested once the Job Card is ready for delivery.", "…does not match the Job Card's vehicle and customer.", "…does not belong to your garage."	sheet shows message	fix input / wait for payment
Delivery already exists	"A delivery has already been requested for this Job Card." (or unique-index violation)	message; screen reloads request	open Track Delivery
Manual delivery blocked	"A delivery trip exists for this Job Card - delivery completes when the driver completes the trip."	message	complete the trip
Trip not arrived	"Driver must arrive at customer before capturing…" (media); `arrive` needs `IN_PROGRESS`	message	press "I Have Arrived"
Evidence missing	"Required {stage} photos are missing: […]"	driver capture screen lists missing shots	capture them
OTP not generated	"No active confirmation code. Ask the customer to generate a new one."	verify screen error	customer opens code screen
OTP expired / exhausted / wrong	"This confirmation code has expired…", "Too many incorrect attempts…", "Incorrect confirmation code."	error text	customer generates a new code
Handover not verified	"Vehicle handover has not been verified yet - ask the customer for the confirmation code."	error on advance	verify first
Pickup completed too early	"Pickup trip must return to garage before completion.", "Mark arrival at the garage before completing the pickup trip."	snackbar	continue leg, arrive
R2 not configured	`MEDIA_STORAGE_NOT_CONFIGURED` (503)	"Media upload is temporarily unavailable." (item `failed`, not retried)	set R2 variables
Signed URL expired	upload PUT/GET fails	queue requests a fresh intent; viewer re-requests access	retry
Mapping of exceptions to HTTP status is in `GlobalExceptionHandler`; Flutter surfaces `ApiException.message` (`friendlyApiErrorMessage`).			

33. Transaction Boundaries

Read from the code (`@Transactional` present unless noted):

- **Customer approval** — `EstimateServiceImpl.approveEstimate(Long)` / `(String)` are `@Transactional`; estimate save, `createRepairTasks` and the job status write commit together.
- **Inspection → estimate preparation** — `ServiceWorkflowServiceImpl.completeInspection(jobCardNumber, list)` is `@Transactional` (inspections + job transition atomic). `createJob`, `prepareEstimate`, `approveEstimate`, `readyForDelivery` in that class are *not* annotated; they delegate to services that are (`JobCardServiceImpl.createJobCard`, `prepareEstimate`, …). The Flutter chain job-create → inspection-start is **two** transactions.
- **Manager confirmation** — `JobCardServiceImpl.startRepair` `@Transactional`.

- **Repair completion** — `RepairTaskServiceImpl.completeRepair`, `JobAssignmentServiceImpl.completeJob` `@Transactional` (task, assignment sync, job aggregate).
- **Delivery trip completion** — `NavigationTripServiceImpl.completeTrip` `@Transactional`; it calls `DeliveryServiceImpl.completeDeliveryFromTrip` (`@Transactional`, joins the outer transaction): trip, request, Delivery row and job status commit or roll back together.
- **Booking → JobCard** — `JobCardServiceImpl.createJobCard` `@Transactional`: job, booking `COMPLETED` and request `jobCardId` are atomic; a validation failure throws before any write.
- **Driver assignment** — `assignDriver` `@Transactional` plus DB unique index (`V42`) for the concurrent-assign race.
- **Media processing** — `MediaProcessingScheduler` is not one transaction: R2 writes are idempotent, the row save precedes the source delete (crash-safe).
- **Notifications (2026-10-02)** — every publish call sits **inside** the business method's own transaction (`NotificationEventPublisher.publish` is `REQUIRED`): outbox row and business change commit or roll back together (`NotificationPipelineIT.businessRollback_leavesNoOutboxRow`). `EstimateServiceImpl.rejectEstimate` was **not** transactional before and received `@Transactional` so its outbox row is atomic with `REJECTED` (the only transactional change). `InvoiceServiceImpl.createInvoice` has no `@Transactional` (pre-existing; the workflow path `generateInvoice` has). Worker steps are short separate transactions; the FCM call runs outside any transaction.

34. Legacy Compatibility Matrix

Component	Current purpose	Why retained	New code should use?	Backward-compatible behaviour	Possible removal
Google Drive media (`GoogleDrive*`, `/content`, `POST /job-cards/{id}/media` multipart)	old media + retry scheduler	rows exist in production	**No** (R2 flow)	served through the proxied content path	after migration of old media
R2 rows with original + thumbnail/evidence	old R2 media	not migrated/deleted	No	`evidence` variant falls back to the original	after an approved migration
Legacy JobCard states `ESTIMATE_APPROVED`, `WORK_COMPLETED`, `INVOICED`, `PAYMENT_PENDING`, `PAYMENT_COMPLETED`	old rows	still readable/movable via validator	**Do not write**	projection/journey map them	when no rows remain
Delivery requests without `jobCardId`	pre-V61	history	not created anymore	trip completes, no job effect	after backfill or cleanup
Duplicate endpoints `/workflow/{n}/…` vs `/jobcards/{n}/…` (repair start, inspection, QC)	two client generations	Flutter uses both	either — they delegate to the same rule	identical behaviour	consolidate later
`JobAssignmentType.DRIVER`, `GET /job-assignments/my/driver`	earlier driver model	no writer/caller found	**No** (use trips)	returns empty	remove
Unused workflow endpoints (`POST /workflow/{n}/delivery`, `/repair/complete`, employee `/estimate/approve`)	compatibility	UI disconnected on purpose	No	`/repair/complete` is check-only	after client audit
Declared-only statuses (`Estimate SENT_FOR_APPROVAL/EXPIRED`, `RepairStatus ON_HOLD/CANCELLED`, `JobAssignment ON_HOLD/QC_*/REWORK`, `NavigationRequest/Trip CANCELLED`, `HandoverStatus CANCELLED`, `DeliveryStatus PENDING/READY_FOR_DELIVERY`)	reserved	DB/enums	not without a feature	never written	prune or implement
`flutter_compress` dependency	no Dart code imports it; Media3 is declared directly	pubspec change needs approval	No	none	remove from `pubspec.yaml`

Component	Current purpose	Why retained	New code should use?	Backward-compatible behaviour	Possible removal
Old media rows without capture metadata	pre-V59	never fabricated	—	footer/metadata sheet omit missing fields	—
`ServiceWorkflowServiceImpl.completeInspection(String)` (returns null)	overload	unused	No (list overload)		remove
`PasswordResetNotificationService` / `LoggingPasswordResetNotificationService`	stub that only logs a warning (no e-mail/SMS delivery)	password-reset flow needs a sender	No (not part of the push system)	token generated, nothing delivered	replace with a real sender when e-mail/SMS is implemented

35. Deployment Runbook

1. Backend: run `mvn test` (expect only the known `InvoiceServiceImplTest` failures) and `mvn -DskipTests package` locally. Optional: `mvn test -Dtest=NotificationPipelineIT` against the isolated local `garageos_test` database.
2. Commit backend (migrations `V60`, `V61`, `V62` included). **Do not** edit any earlier migration.
3. Deploy the backend on Render (Docker build runs `./mvnw clean package -DskipTests`).
4. On startup Flyway applies pending migrations (V60/V61/V62) before JPA `validate`. Check the Render logs for "Successfully applied" and that the app starts (a validate error means a migration/entity mismatch).
5. Verify in PostgreSQL: `select version, success from flyway_schema_history order by installed_rank desc limit 3;` and `\d navigation_request` shows `job_card_id` and the two indexes.
6. Verify R2 variables exist in Render (names in §36); an unset set yields `MEDIA_STORAGE_NOT_CONFIGURED` on upload.
7. Health: `GET /api/v1/health`.
8. API smoke: authenticated `GET /api/v1/workflow/{n}/status` on a job in `REPAIR_PENDING` returns `completedSteps` **without** `APPROVAL`.
9. Build Flutter: `flutter analyze`, `flutter test`, then the release build (`flutter build appbundle` — needs the local signing files; not part of this handbook's validation).
10. Publish the app **after** the backend (older apps keep working against the new backend; the new app needs V61 + the new delivery-request validation).
11. Manual smoke test: §38 (at least WF-004/005, PK-004/005, DL-001…006).
12. **Notifications (optional, config-dependent):** set `NOTIFICATIONS_ENABLED=true` and `FIREBASE_CREDENTIALS_BASE64` in Render, redeploy, check the log line `[NOTIF_FCM] Firebase sender initialised.` and `[NOTIF_OUTBOX] Worker started`; then run §38 `NT-*`. The Flutter build needs the per-environment `android/app/google-services.json` (git-ignored). **Rollback (notifications):** set `NOTIFICATIONS_ENABLED=false` and restart — publisher becomes a no-op, the worker never starts, no FCM calls; tables stay. **Rollback:** redeploy the previous backend build — safe against the V60/V61 schema (additive, nullable/defaulted). Old *apps* cannot create delivery requests against the new backend (no `jobCardId`/coordinates → rejected). Data rollback (dropping `job_card_id`) needs a new migration and loses the delivery↔job link; delivery trips completed by the new code have already written Delivery rows and `DELIVERED` job status, which a code rollback does not undo.

36. Render, Cloudflare R2 and Environment

- **Render:** Docker web service from the backend `Dockerfile`, PostgreSQL database; `server.port=${PORT:8080}`; production URL `https://garagest-backend.onrender.com`.
- **R2:** private bucket; no public URLs; every access is a presigned URL from `R2MediaStorageProvider`. Object naming §23. Trip media is **not** in R2.
- **Environment variable names only:** `DB_HOST DB_PORT DB_NAME DB_USERNAME DB_PASSWORD PORT` · `R2_ACCOUNT_ID R2_ACCESS_KEY_ID R2_SECRET_ACCESS_KEY R2_BUCKET_NAME R2_UPLOAD_URL_TTL_MINUTES R2_PLAYBACK_URL_TTL_MINUTES` · `GOOGLE_MAPS_ROUTES_API_KEY GOOGLE_MAPS_PLACES_API_KEY` · `GOOGLE_DRIVE_APPLICATION_NAME GOOGLE_DRIVE_CLIENT_ID GOOGLE_DRIVE_CLIENT_SECRET GOOGLE_DRIVE_REDIRECT_URI GOOGLE_DRIVE_ROOT_FOLDER_ID` (legacy) · **notifications:** `NOTIFICATIONS_ENABLED` (default `false`) and `FIREBASE_CREDENTIALS_BASE64` (Base64 of the Firebase service-account JSON; Render secret, never committed). Optional tuning keys in `application.properties`: `notifications.worker.delay-ms`, `batch-size`, `max-attempts`, `stale-lock-minutes`, `base-backoff-seconds`. JWT

settings live under `security.jwt.*` (see §7 limitation). Flutter: `EnvConfig` via `--dart-define`; Android Maps key via local, gitignored Gradle properties. Notifications: `android/app/google-services.json` (per environment, git-ignored; the Google Services plugin is applied only when it exists).

37. Testing Strategy and Test Evidence

Strategy: backend Mockito service tests (`mvn test`); `ServiceWorkflowServiceImplCompleteInspectionRollbackIT` and `NotificationPipelineIT` need a local PostgreSQL (`garageos_test`) and are not in the default run (`mvn test -Dtest=…IT`). Flutter: `flutter analyze`, `flutter test` (unit + widget), `flutter build apk --debug`. GPS, maps, STOMP, native video, OTP UX, R2 and **real push delivery** need **manual** device tests (§38).

Key tests for the 2026-09-30 change: `ManagerConfirmationGateTest`, `NavigationTripLifecycleTest`, `NavigationRequestDeliveryTest`, `DeliveryCompletionTest`, `JobCardBookingLinkTest`, additions in `RepairTaskServiceImplJobAssignmentSyncTest` and `CustomerVehicleJourneyServiceImplTest`; Flutter `workflow_approval_gate_test.dart`, `assign_driver_test.dart`. Media: `EvidenceOnlyMediaModelTest`, `MediaKeysAndEvidenceTextTest`, `media_upload_queue_video_evidence_test.dart`, `video_viewport_test.dart`, `media_evidence_text_test.dart`.

Notification tests (2026-10-02) — AUTOMATED VALIDATION (stub FCM sender / fake gateway; no real Firebase):

Area	Tests	What they prove
Publisher / idempotency	`NotificationEventPublisherTest` (7)	disabled = no-op; stable event key; same event twice → same key; actor captured; malformed ignored
Recipients / tenant isolation	`NotificationRecipientResolverTest` (13)	customer with/without account, mobile-matched staff not a customer, technician/driver/manager/SA/owner, actor suppression, de-duplication, **cross-garage rejection**, QC-failure technicians from facts
Templates	`NotificationTemplateFactoryTest` (4)	every event × persona renders within column limits; no PII; persona wording
Device tokens	`DeviceTokenServiceImplTest` (7)	register, update, **reassignment to another user**, unregister ownership
History API	`NotificationServiceImplTest` (5)	paging clamp, unread count, mark read idempotent, **other user's notification not found**
Outbox / retry / FCM results	`NotificationOutboxProcessorTest` (15), `NotificationOutboxWorkerTest` (4)	claim + stale recovery, fan-out per device, replay without duplicates, backoff and bounds, success / transient / **invalid token → deactivated** / configuration error pauses the tick
Wiring	`EstimateNotificationWiringTest` (4)	`rejectEstimate` is `@Transactional`, publishes `ESTIMATE_REJECTED`, nothing on a rejected re-reject, works without a publisher
Real database	`NotificationPipelineIT` (6, PostgreSQL `garageos_test`)	**business rollback leaves no outbox row**; commit + replay de-duplicated; full chain outbox → notification → delivery → FCM stub; invalid token; transient failure stays `PENDING`; user-scoped history/mark-read
Flutter	`push_payload_and_resolver_test`, `notification_navigation_service_test`, `push_notification_service_test`, `notification_service_and_state_test`, `auth_state_logout_hook_test` (50 together)	payload parsing, **role-aware routing**, **cold-start queue** (not dispatched before session/splash), 404 ownership, duplicate-tap suppression, token registration/refresh, **logout ordering** (real `AuthState`: hook before session clear, hook failure/hang never blocks logout), foreground de-duplication, API contract (PATCH, token in body)

Evidence (commands executed from each repo root on 2026-10-02):

Check	Command	Result
Backend tests	`./mvnw test`	**FAIL (known, unrelated)** — 226 run, 2 failures + 1 error, all in `InvoiceServiceImplTest.receivePayment_*` (identical with the notification change reverted); every other test passes, including all 59 notification unit tests
Backend integration	`./mvnw test -Dtest=NotificationPipelineIT`	**PASS** — 6/6 tests
Backend package	`./mvnw clean package -DskipTests`	PASS

Check	Command	Result
Flutter analyze	`flutter analyze`	**PASS** — 0 errors, 2 warnings (pre-existing unused imports), 94 info-level lints; no issues in the notification code
Flutter tests	`flutter test`	**PASS** — 368 tests
APK	`flutter build apk --debug`	PASS (also installed and run on a physical device by the product owner)
AAB / release build	—	NOT RUN
V61 on PostgreSQL	—	ENVIRONMENT REQUIRED (not executed)
V62 on PostgreSQL	applied by Flyway when `NotificationPipelineIT` started the Spring context	PASS (isolated `garageos_test` database)
Physical Android validation of notifications	manual (§28A.15, §38 `NT-001…004`)	**PHYSICALLY VALIDATED — ANDROID** (partial scope; see §28A.15)
Device flows (§38, other)	—	MANUAL REQUIRED
`start_repair` projection filter	—	no automated test (NOT COVERED)
`DeliveryScreen` restoration	—	no widget test (NOT COVERED); logic exercised via service tests only
Known unrelated failures: 3 in `InvoiceServiceImplTest` (`receivePayment_*`), present before this work. **Automated tests and the physical device test are different evidence:** automated tests do not contact Firebase.		

38. Manual Test Matrix

All rows are **manual/device required** unless stated. Roles: M manager, T technician, D driver, C customer.

ID	Scenario	Role	Precondition	Steps	Expected result	Backend state
WF-001	Job creation	M	customer + vehicle exist	+ Workflow → job step → complaints, odometer, date → save	Inspection step opens	JobCard `INSPECTION_PENDING`
WF-002	Inspection	M	WF-001	save all inspections	Estimate step	`INSPECTION_COMPLETED` → `ESTIMATE_PENDING`
WF-003	Estimate	M	WF-002	create, add items, summary → proceed	Approval step "waiting"	`WAITING_FOR_APPROVAL`, estimate `WAITING_FOR_APPROVAL`
WF-004	Customer approval	C→M	WF-003	customer approves; manager opens/refreshes	Approval shows "Customer Approved — Manager Confirmation Required"; Repair NOT started	Estimate `APPROVED`, tasks `PENDING`, job `REPAIR_PENDING`; `completedSteps` lacks `APPROVAL`
WF-005	Manager confirmation	M	WF-004	tap Proceed to Repair	Repair step	`REPAIR_IN_PROGRESS`
WF-006	Technician repair	T	WF-005, task assigned	before WF-005: try start (must fail); after: accept, start, complete	start blocked then allowed; last task → QC next	tasks `IN_PROGRESS/COMPLETED`; job `REPAIR_COMPLETED`
WF-007	QC	M	WF-006	proceed to QC → pass	Invoice step	`QUALITY_CHECK` → `READY_FOR_INVOICE`

ID	Scenario	Role	Precondition	Steps	Expected result	Backend state
WF-008	Invoice/payment	M	WF-007	generate invoice; customer accepts; pay	Delivery step	`INVOICE_GENERATED` → `READY_FOR_DELIVERY`
PK-001	Pickup creation	C,M	garage exists	book with pickup; manager confirms; assign driver	driver sees trip	request `ASSIGNED`, trip `ASSIGNED`
PK-002	Pickup navigation	D	PK-001	accept, start, open map	route to customer; customer sees driver	trip `IN_PROGRESS`, request `IN_PROGRESS`
PK-003	Pickup OTP	D,C	arrived + 5 photos	customer opens code; driver enters	"Pickup confirmed"	handover `VERIFIED`
PK-004	Pickup return	D	PK-003	Start Return to Garage; navigate; arrive	route to the **garage**; no OTP/evidence asked	leg `CUSTOMER_TO_GARAGE`, `arrivedAt` set
PK-005	Pickup completion	D,M	PK-004	Complete Pickup Trip; then Create Job Card from booking	trip completed; job linked	trip/request `COMPLETED`; `JobCard.bookingId` set, Booking `COMPLETED`, request `jobCardId` set
DL-001	Delivery request	M	job `READY_FOR_DELIVERY`	Delivery → Request Delivery	sheet opens; reopening step restores state	request `REQUESTED` (`jobCardId` set)
DL-002	Delivery map	M	DL-001	Select on map; confirm	address + lat/lng shown	request has address, lat, lng
DL-003	Driver assignment	M,D	DL-002	assign driver; driver accepts	driver sees delivery trip	trip `ASSIGNED` → `ACCEPTED`; request `ASSIGNED`
DL-004	Delivery navigation	D,C	DL-003	start; navigate	marker at chosen delivery point; customer tracks	trip `IN_PROGRESS`
DL-005	Delivery OTP	D,C	arrived + 5 delivery photos	customer opens code; driver verifies	"Delivery confirmed"	handover `VERIFIED` (DELIVERY)
DL-006	Delivery completion	D,M	DL-005	Complete Delivery; manager reopens step; Close Job	job delivered then closed; manual delivered button absent	trip/request `COMPLETED`; Delivery `DELIVERED`; job `DELIVERED` → `CLOSED`
NAV-001	STOMP	D,C	active trip	move >10 m	customer marker moves smoothly	rows in `driver_location_history`
NAV-002	REST fallback	C	active trip	drop socket / airplane toggle	marker updates ≈ every 10 s	—
MED-001	Image	M/T	job open	capture image with GPS	evidence footer incl. address, lat/lng, accuracy; thumbnail in grid	R2: `_evidence.jpg`, `_thumbnail.jpg` only; `evidence_only=true`
MED-002	Video	M/T	job open	record portrait + landscape	evidence burned in; correct size; thumbnail	R2: `_evidence.mp4`, `_thumbnail.jpg`
MED-003	Customer playback	C	MED-001/002	open media	evidence shown, video plays; repeat open uses cache	—
MED-004	Legacy media	M,C	old media exist	open old image/video	still displays	unchanged rows
NT-001	Foreground push	C/M	notifications enabled, app open	complete a workflow step that notifies (e.g. generate invoice)	local notification shown, badge increments	outbox `SENT`, delivery `SENT` — **PHYSICALLY VALIDATED (Android; owner-observed)**

ID	Scenario	Role	Precondition	Steps	Expected result	Backend state
NT-002	Background / tray	C/M	app backgrounded	trigger an event	tray notification with sound; vibrate/silent per device mode	**PHYSICALLY VALIDATED (Android)**
NT-003	Tap + deep link	C/M	NT-002	tap the notification	GarageST opens on the correct screen (job-card context shown)	**PHYSICALLY VALIDATED (Android)** — *Vehicle Ready for Delivery* and *Payment Received* observed
NT-004	History + badge	C/M	NT-001	open Notifications; mark one / all read	list, unread dot, badge update	**PHYSICALLY VALIDATED (Android)**
NT-005	App killed (cold start)	each persona	app force-killed	trigger event; tap notification	app starts, session restores, correct persona screen opens	PENDING (not individually recorded)
NT-006	Permission denied	any	deny Android 13+ permission	trigger event	no tray notification; history/badge still work	PENDING
NT-007	Multi-device	any	same account on two phones	trigger event	both receive; one invalid token does not block the other	PENDING
NT-008	Logout / account switch	any	A logged in, token registered	log out; log in as B; trigger an event for A	A's phone receives nothing; token reassigned/deactivated	PENDING
NT-009	Per-persona events	T,D,O	assignments/trips exist	trigger `REPAIR_TASK_ASSIGNED`, `HANDOVER_CODE_GENERATED`, `QUALITY_CHECK_FAILED`, owner events	recipients per §28A.4	PENDING (not every one of the 32 events was individually exercised)
NT-010	Customer without app account	M	customer with no user account	trigger a customer event	no push, no error	PENDING

39. Debugging Runbook

Approval screen skipped: `GET /workflow/{n}/status` → is `completedSteps` containing `APPROVAL` while `status = REPAIR_PENDING`? (should not). Check `JobCard.status`, `Estimate.status`. If backend is right, inspect `WorkflowController._onWorkflowStateChanged` (debug log "WORKFLOW RESOLVE") and any screen calling `nextStep()` after a refresh. **Technician cannot start repair:** JobCard status (must be `REPAIR_IN_PROGRESS`), RepairTask `ASSIGNED`, JobAssignment `ACCEPTED`, manager confirmation done? Error text says which. **Pickup stuck:** `TripStatus`, `TripLeg`, `arrivedAt` (reset by `continue`), `vehicle_handover` (status/expiry/`failed_attempts`), photos per stage (`navigation_trip_media` shot types), `NavigationRequest.status`. On the return leg the driver needs "I Have Arrived" then "Complete Pickup Trip". **Delivery doesn't appear / can't be requested:** `navigation_request` row (`request_type='DELIVERY'`, `job_card_id`, status), `GET /navigation/requests/job-card/{id}/delivery`, job status `READY_FOR_DELIVERY`, address+coordinates present, unique index violations, `DeliveryScreen` reload (`_loadDeliveryRequest`). **Delivery completed but job not DELIVERED:** request has `job_card_id`? (NULL = legacy); invoice `GENERATED` + `PAID`?; a `delivery` row already exists?; transaction rolled back (trip stays `IN_PROGRESS`). **Navigation not updating:** location permission + service enabled; driver has `ActiveTripScreen` open; STOMP connected and subscribed to `/topic/trips/{id}/location`; interceptor authorization; REST fallback (`GET …/location`); distance filter (10 m) — a stationary driver publishes nothing; route recalculation only after 150 m. **Media not loading:** R2 variables; upload intent response (`storageKey`); `/complete` called; row `upload_status` / `retry_count` / `last_error`; `evidence_key` set (images render asynchronously, `available=false` until then); presigned URL age (TTL 10 min); Flutter cache key (`media-video-{id}`); for video, `thumbnailUploaded` and thumbnail object existence. **Migration/startup failure:** `ddl-auto=validate` mismatch → check `flyway_schema_history` and entity columns. **Notification not received:** `NOTIFICATIONS_ENABLED` true? `FIREBASE_CREDENTIALS_BASE64` present and the log shows `[NOTIF_FCM] Firebase sender initialised.`? `notification_outbox` row exists and `status` (PENDING = worker not running/not due, FAILED = see `last_error`)? `notification` row created for the user (no row → recipient not resolved: check garage, role, customer↔user mobile match; actor suppression)? `notification_delivery` row (none → user has no active `device_token`, or push unavailable at fan-out)? delivery `INVALID_TOKEN` → the app must re-register (log in again). Android: notification permission and channel settings. App side: `google-services.json` present for that build (`FirebaseBootstrap.available`). **Duplicate or missing notification after a repeated transition:** compare `event_key` values — the discriminator must differ per genuine pass (§28A.7). **Tap opens nothing:** the notification may belong to another user (mark-read 404 by design); the app may still be on splash/login (taps are queued); unknown persona; missing ids → history screen.

40. Architectural Rules — DO NOT BREAK THESE

1. JobCard ≠ RepairTask; task work never writes job state except the all-tasks-complete aggregate.
2. Customer approval ≠ manager confirmation; only `JobCardServiceImpl.startRepair` moves `REPAIR_PENDING → REPAIR_IN_PROGRESS`.
3. Backend business state is authoritative; Flutter derives steps from `completedSteps` and never advances locally past a status-refreshing mutation.
4. Every writer of `JobCard.status` goes through `JobCardStatusValidator`.
5. A delivery completes through its delivery trip; the manual `/deliveries` record must not contradict it.
6. Trip/delivery state must not live only in Flutter screen state.
7. Direct-to-R2 media upload; no Spring media proxy; do not raise the multipart limit.
8. New media = evidence + thumbnail, no permanent original; legacy media stays compatible; no destructive migration without approval.
9. No secrets in source control; never document values.
10. Never modify an applied Flyway migration.
11. Keep `JobCardMedia` and `NavigationTripMedia` separate.
12. Do not reintroduce a garage-side OTP or a second `continueTrip` on the return leg.
13. Notification events are published **inside** the business transaction through `NotificationEventPublisher` and only from meaningful transitions; never call FCM from a business transaction and never add DB triggers or "watch every update" logic.
14. Every staff notification recipient must belong to the event's garage; keep recipient rules in `NotificationRecipientResolver` only; never use FCM topics as authorization.
15. The FCM payload carries identifiers only; notification text carries no customer/vehicle/financial data; never log FCM tokens or credentials.
16. Logout must unregister/delete the device token **before** the session is cleared (`AuthState.onBeforeLogout`); notification cleanup must never block logout.
17. Adding or changing a notification event requires updating the matrix in §28A.4, the resolver, the template factory and the tests.

41. Architecture Decision Records

ADR-001 — Direct-to-R2 media upload

Status: Accepted (implemented). **Context:** Drive-backed uploads buffered whole files in the JVM and had no range support. **Problem:** large photos/videos through Spring exhaust memory and hit the multipart limit. **Decision:** presigned PUT to a private R2 bucket + persistent Flutter queue; backend verifies at `/complete`. **Alternatives:** raise multipart limit (rejected); proxy streaming through Spring (rejected: cost/latency); keep Drive. **Why:** Spring never touches bytes; resumable client queue. **Consequences:** client owns retry; orphan rows on URL expiry; R2 config needed. **References:** `MediaServiceImpl.createUploadIntent/completeUpload`, `R2MediaStorageProvider`, `MediaUploadQueueService`.

ADR-002 — Evidence is the canonical new media

Status: Accepted. **Context:** original + evidence + thumbnail tripled storage and exposed unstamped originals. **Problem:** need tamper-evident, single asset. **Decision:** store only evidence + thumbnail; images rendered server-side from a temporary source, videos rendered on-device (Media3). **Alternatives:** keep originals; FFmpeg (APK size, license); server video processing (no decoder on Render). **Why:** Media3 already in the APK; one file to serve. **Consequences:** two renderers kept in sync by test literals; `evidence_only` flag; legacy fallback. **References:** `MediaProcessingScheduler`, `MediaEvidenceText`, `EvidenceVideoProcessor.kt`, `media_evidence_text.dart`.

ADR-003 — JobCard state vs RepairTask state

Status: Accepted. **Context:** technician actions were flipping the job. **Problem:** task state substituted for job state. **Decision:** separate machines; task start/complete require the job to be `REPAIR_IN_PROGRESS`; the only coupling is the automatic all-complete aggregate. **Alternatives:** derive job state from tasks (rejected). **Why:** manager gate must be unbypassable. **Consequences:** technicians blocked until confirmation. **References:** `JobAssignmentServiceImpl.startJob`, `RepairTaskServiceImpl.startRepair/completeRepair`.

ADR-004 — Customer approval vs manager confirmation

Status: Accepted. **Context:** approval landed the job at `REPAIR_PENDING` and the projection marked `APPROVAL` complete. **Problem:** manager skipped the Approval screen. **Decision:** keep `REPAIR_PENDING` as the "approved, awaiting manager" state; complete `APPROVAL` only at `REPAIR_IN_PROGRESS`; confirmation stays `POST /workflow/{n}/repair/start`. **Alternatives:** new `MANAGER_CONFIRMATION_PENDING` status (more migration/validator/legacy churn, rejected); client-side flag (violates backend authority). **Why:** smallest consistent change. **Consequences:** `resolveCompleted` is the single place encoding the distinction; screens must not `nextStep()` after refresh. **References:** `ServiceWorkflowServiceImpl.resolveCompleted`, `JobCardServiceImpl.startRepair`, `ApprovalScreen`.

ADR-005 — Trip ↔ JobCard linkage

Status: Accepted. **Context:** trips/requests referenced only vehicle/customer/garage; the journey guessed by vehicle. **Decision:** one nullable `navigation_request.job_card_id` (delivery = job delivered; pickup = job created from its booking). **Alternatives:** FKs on trips and bookings; new join table; backfill legacy rows (rejected as unnecessary). **Why:** minimal, no duplicate relationship. **Consequences:** legacy rows NULL; no FK constraint; partial unique index. **References:** V61, `NavigationRequestServiceImpl`, `JobCardServiceImpl.createJobCard`, `CustomerVehicleJourneyServiceImpl`.

ADR-006 — Delivery trip completion is the delivery completion

Status: Accepted. **Context:** `POST /deliveries` and trip completion were independent. **Problem:** contradictory completion paths. **Decision:** completing a verified delivery trip creates the Delivery row and moves the job to `DELIVERED` in one transaction; manual delivery only when no delivery request exists. **Alternatives:** trip completion informational only; remove `/deliveries`. **Why:** OTP + evidence should be what proves delivery. **Consequences:** requests need `jobCardId`; legacy requests don't affect jobs. **References:** `NavigationTripServiceImpl.completeTrip`, `DeliveryServiceImpl.completeDeliveryFromTrip/createDelivery`.

ADR-007 — Map-based delivery location

Status: Accepted. **Context:** delivery address was free text with optional coordinates. **Decision:** reuse `LocationPickerScreen`; require address + latitude + longitude (address falls back to the coordinates). **Alternatives:** second picker; keep text-only. **Why:** navigation needs coordinates; one picker to maintain. **Consequences:** old apps can no longer create delivery requests. **References:** `CreateDeliveryRequestSheet`, `NavigationRequestServiceImpl.validateDeliveryRequest`.

ADR-008 — Firebase FCM + PostgreSQL Transactional Outbox Notification Architecture

Status: Accepted (implemented 2026-10-02; PHYSICALLY VALIDATED — ANDROID). **Context:** workflow steps (estimate ready, repair done, invoice, vehicle ready…) needed to reach customers and staff when the app is closed. The system already had STOMP for live location, a single Render instance, PostgreSQL, and one DB-polling scheduler (media retry). No broker existed. **Decision:** emit a business event from the transition inside the business transaction into a **PostgreSQL outbox**; a background worker resolves recipients (tenant-scoped), writes per-recipient history rows and per-device delivery rows, then sends through **Firebase Cloud Messaging** (Admin SDK, HTTP v1). Flutter shows foreground pushes with local notifications, lets the system tray show background ones, and routes taps through a queued, role-aware navigation service. History lives in PostgreSQL; FCM is delivery only. **Alternatives considered:** (1) call FCM from the business method — rejected: an FCM outage would fail or slow business operations; (2) Spring `@TransactionalEventListener` only — rejected as sole mechanism: events are lost on a crash between commit and handler; (3) Kafka/RabbitMQ — rejected: extra infrastructure for a single-instance, low-volume system; the outbox gives the needed durability; (4) database triggers / watch every UPDATE — rejected: notifications must follow business meaning, not column changes; (5) FCM topics per garage — rejected: topics are not an authorization system; (6) WebSocket for notifications — rejected: no per-user destinations, only connected while a trip screen is open. **Why chosen:** atomic with the business change (no phantom or lost notification), independent retry, reuses the existing polling precedent, keeps PostgreSQL the source of truth, minimal new infrastructure. **Consequences:** at-least-once delivery (client de-duplicates by notification id); up to ~5 s latency; event keys need a transition discriminator; recipient rules centralised; Firebase service-account secret to manage; iOS requires further work. **Failure and retry:** outbox/delivery retried with exponential backoff (30 s doubling, cap 1 h, 6 attempts); stale `PROCESSING` rows recovered after 5 min; `UNREGISTERED` tokens deactivated; credential errors pause sending and never affect business transactions; disabled/no-credentials mode keeps history only. **Security considerations:** identity from JWT; garage isolation in the resolver; identifier-only payload; ownership check on tap (mark-read 404); credentials only via environment; no tokens in logs. **References:** `modules/notification/**`, `V62__create_notification_system.sql`, `NotificationOutboxWorker`, `NotificationRecipientResolver`, `lib/core/push/**`, `lib/features/notification/**`, §28A.

42. Change Log

2026-10-02 — Push Notification System — Firebase FCM + Transactional Outbox

- Backend `modules/notification`: 32 business events (verified against `NotificationEventType`) published from the service transitions inside the business transaction; recipient resolver with garage isolation and actor suppression; persona-specific templates.
- Database **V62**: `device_token`, `notification`, `notification_outbox`, `notification_delivery` with idempotency constraints.
- Transactional outbox + retry worker (own executor, `SKIP LOCKED`, backoff, stale-lock recovery); **Firebase Admin SDK 9.9.0**; no-op sender when disabled/unconfigured (`NOTIFICATIONS_ENABLED`, `FIREBASE_CREDENTIALS_BASE64`).
- APIs: device register/unregister, history, unread count, mark read, mark all read. `EstimateServiceImpl.rejectEstimate` made `@Transactional`.
- Flutter: `firebase_core` / `firebase_messaging` / `flutter_local_notifications`; foreground local notifications, background tray notifications, cold-start queue, role-aware deep links, notification history screen, unread bell badge in the five home shells, device-token lifecycle incl. ordered logout cleanup (`AuthState.onBeforeLogout`), `ApiClient.patch`.
- Android: Google Services plugin (conditional on `google-services.json`), desugaring, channels `garagest_urgent/default/low`, runtime permission flow.
- **Validation:** automated — backend 226 tests (223 pass; 3 pre-existing `InvoiceServiceImplTest` failures), `NotificationPipelineIT` 6/6, Flutter 368 pass, backend jar and Android debug APK built; **physically validated on Android** (sound, vibration per device mode, tray, history, badge, tap + deep link; *Vehicle Ready for Delivery* and *Payment Received* observed with job-card context).
- **Known limitations:** iOS not implemented; customer match by mobile number; legacy backend logout does not revoke sessions; launcher icon reused as notification icon; stub FCM in automated tests; `createInvoice` non-transactional; no retention job or preferences (§28A.16). **2026-09-30 — Workflow + trip/navigation architecture**
- Approval: `APPROVAL` completes only at `REPAIR_IN_PROGRESS`; approval screen shows manager-confirmation state; `resolveStep` aligned to the shell numbering.
- Repair: technician start no longer moves the job; task completion requires `REPAIR_IN_PROGRESS`; `start_repair` action manager-only.
- Pickup: return-leg completion (arrive + complete), OTP refused on the return leg, request `IN_PROGRESS/COMPLETED` written.
- Booking ↔ JobCard: `bookingId` sent from the workflow (Booking screen → Create Job Card); pickup request linked to the job.
- Delivery: request tied to a job (V61) with address + coordinates, rediscovery endpoint, trip completion creates the Delivery and marks the job `DELIVERED`, manual delivery guarded; Delivery screen restores from the backend.
- Map: delivery location selected with `LocationPickerScreen`.
- Journey: linked pickup trips no longer hide the job-card stage.
- Validation: see §37. **2026-09-30 (earlier) — Media:** evidence + thumbnail only, on-device video burn-in, orientation-correct output (`V60`).

43. How to Maintain This Handbook

1. Read the relevant section before changing architecture.
2. Inspect the actual code before changing documented behaviour — if they disagree, the code wins and the handbook must be fixed.
3. Update this handbook in the same task as any architectural change.
4. Add or amend an ADR for every significant decision (status, alternatives, consequences, references).
5. Update the state-machine tables (§9) whenever a transition, writer or guard changes.
6. Update the API reference (§27) when a business endpoint, role or validation changes.
7. Add recurring problems to the debugging runbook (§39) and error table (§32).
8. Never document secrets — names of variables only, no values (including defaults found in property files).
9. Never document unimplemented behaviour as current; mark *Declared-only* / *Not implemented* explicitly.
10. Keep legacy behaviour labelled in §34 and record removal criteria.
11. When notification events, recipients or API change, update §28A (matrix, recipients, lifecycle, limitations) and the `NT-*` test rows in the same task.